Tournament Heaps

Telman Utepov

ID 260462729

McGill University, Montreal

May, 2015

A thesis submitted to McGill University in partial fulfillment of the requirements of the degree of Master of Computer Science

© Telman Utepov 2015

Table of Contents

Abstract 3
Acknowledgements 4
Chapter 1. Introduction 5
  Motivation 9
Chapter 2. Tournament Trees 12
  Description 12
  Operations on Tournament Trees 13
  Insert 14
  DeleteMin 15
  MakeTree and Tournament Sort 16
Chapter 3. Tournament Heaps 17
  Introduction 17
  Structure 18
    Internal and leaf nodes 19
    Internal node 19
    Leaf node 19
  Rules 21
  Operations 21
    Insert 22
    Merge 23
    DeleteMin 23
    DecreaseKey 28
    Delete 30
Chapter 4. Analysis 31
  Ranks and Tree Sizes 31
  Verification of the amortized analysis claims 33
    Insert 33
    Merge 33
    DecreaseKey 33

DeleteMin 33
Delete 34
Chapter 5. Comparison with Fibonacci heap 35
Similarities 35
Differences 35
Simulation 37
Conclusion 42
References 43
Appendix A 45
Listing of programming code 45
Tournament heap 45
Fibonacci heap 52

Abstract

Tournament trees are valid replacements for binary heaps as priority queues, as they are optimal for sorting. Their generalization, the tournament heap, is shown to be a competitive replacement for the Fibonacci heap in an amortized sense.

Keywords: tournament heap, priority queue.

Les arbres tournoi sont valables pour le remplacement des tas binaire comme des files de priorité, puisqu'ils sont optimaux pour le tri. Leur généralisation, le tas tournoi, se révèle être un remplacement concurrentiel pour le tas Fibonacci dans un sens amorti.

Mots-clés: tas tournoi, file de priorité.

Acknowledgements

I would like to express my deepest appreciation and thanks to my thesis supervisor, Professor Luc Devroye, McGill University, School of Computer Science. I would like to thank you for guiding me through all the steps of developing the idea, elaborating the analysis, and helping with programming of experiment tests, and guiding through writing the report.

Special thanks to my family who encouraged me during the time of doing this thesis work.

Chapter 1. Introduction

The priority queue, as described by Goodrich and Tamassia[1], is an abstract data type that operates on a set of elements that process a comparable key, such as an integer or a float. For abstract data types in general, see Cormen et al[2]. A good description of heaps and computation of the algorithms complexity see Tarjan[3]. The rich source of valuable algorithms and data structures is represented by C. L. Kuszmaul [4].

Priority queues support following operations:

- `Create()`: Creates and returns an empty priority queue.
- `Insert(h, k)`: Creates a new node with key k and inserts it into a priority queue h.
- `GetMin(h)`: Returns the node of a priority queue that contains the minimal key value.
- `DeleteMin(h)`: Deletes the node with minimal key value from the priority queue and returns it.
- `Merge(h1, h2)`: Returns the heap that is a union of priority queues h1 and h2.
- `DecreaseKey(h, x, d)`: Decreases the key value of a node x of a priority queue h by the number of d.
- `Delete(h, x)`: Deletes the node x from priority queue h.

We will use next definitions:

- **Child**: The child of the tree node is the node that lower in a tree hierarchy than the current node and to which node the current node has a reference. Children of the node are nodes that are on the same hierarchy level and referenced by the current node.
- **Rank**: In the tree, the rank of element is its position in the linear order of the tree. The rank of a binomial heap node is the number of its children.

The most well-known implementations of a priority queue are:

- **Binary heap** (Williams, 1964)[5]. This heap is constructed as a binary tree with two constraints: 1. **The shape property:** the binary tree is complete, that is, all levels of the tree, except possibly the last one (deepest) are fully filled, and, if the last level of the tree

is not complete, the nodes of that level are filled from left to right. 2. **The heap property:** keys are increasing on paths away from the root.

- **Binomial heap** (Vuillemin, 1978)[6]. This heap is similar to a binary heap, but supports quick merge. This is a collection of binomial trees. The binomial tree is a tree constructed by the melding of two equal rank trees, where the least root key becomes the root of the new tree, and the root of the second tree is inserted as a first child of the root. So the rank of the resulting tree becomes r, where r-1 is a rank of its two trees from which it is composed.
- **Fibonacci heap** (Fredman and Tarjan, 1987)[7]. This is a collection of binomial trees, similar to Binomial heap, but providing a better amortized running time. Fibonacci heaps were developed by Michael L. Fredman and Robert E. Tarjan in 1984 and first published in a scientific journal in 1987. The name of Fibonacci heap comes from Fibonacci numbers which are used in the running time analysis. As the current research takes a Fibonacci heap as a basis for a new structure, as the most efficient data structure, the structure and special features of the Fibonacci heap will be discussed in detail later.
- **Tournament tree** (Devroye, 2014)[8]. This is a complete binary tree consisting of leaves, containing keys, and keyless internal nodes that keep organizing nodes into an efficient structure somehow initiating a tennis tournament. As the current research is based on tournament trees, the details of its structure and functionality will be described further in the text in the next chapter.

Table 1 compares different types of data structures in terms of efficiency of main operations: `Insert, Merge, GetMin, DeleteMin, Delete`, and `DecreaseKey` is represented below. This table also includes the efficiency of the **tournament heap**, the new data structure introduced in this work. Here **n** and **m** are integers meaning the number of elements in corresponding priority queues h_n, h_m.

Table 1. Comparison of the worst-case time for different data structures.

Data Structure	**Insert**	**Merge (h_n, h_m)**	**Get Min**	**Delete Min**	**Delete**	**Decrease Key**

Binary heap	O(log n)	O(log n * log m)	O(1)	O(log n)	O(log n)	O(log n)
Binomial heap	O(log n)	O(log n + log m)	O(1)	O(log n)	O(log n)	O(log n)
Fibonacci heap (amortized)	O(1)	O(1)	O(1)	O(log n)	O(log n)	O(1)
Tournament tree (worst case)	O(log n)	O(log n * log m)	O(log n)	O(log n)	O(log n)	O(log n)
Tournament heap (amortized)	O(1)	O(1)	O(1)	O(log n)	O(log n)	O(1)

The table shows that the most efficient amortized data structure is the Fibonacci heap. However, its performance is only guaranteed in an amortized sense.

The binary heap and the tournament tree are simpler structures than the binomial and Fibonacci heaps. Every node of these data structures contains two children at most, whereas nodes of binomial and Fibonacci heaps may contain a different number of children, where each child has a different rank.

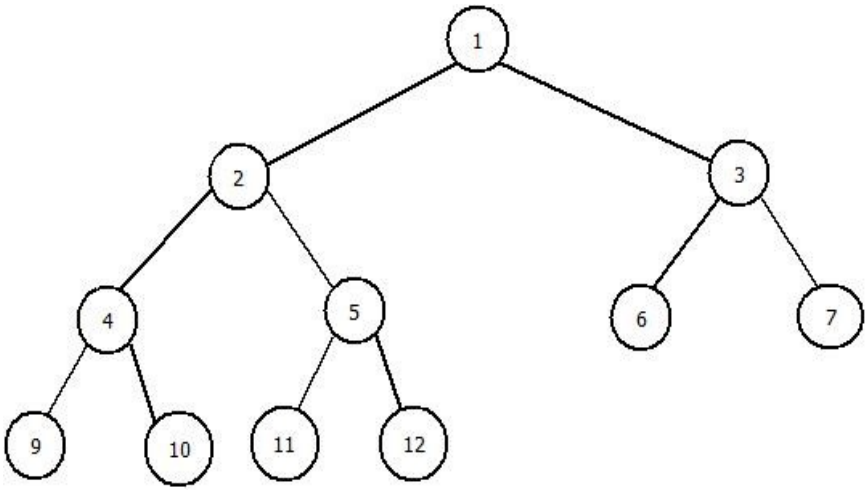


Figure 1. Binary heap.

An example of a binomial or Fibonacci heap is represented in Figure 2.

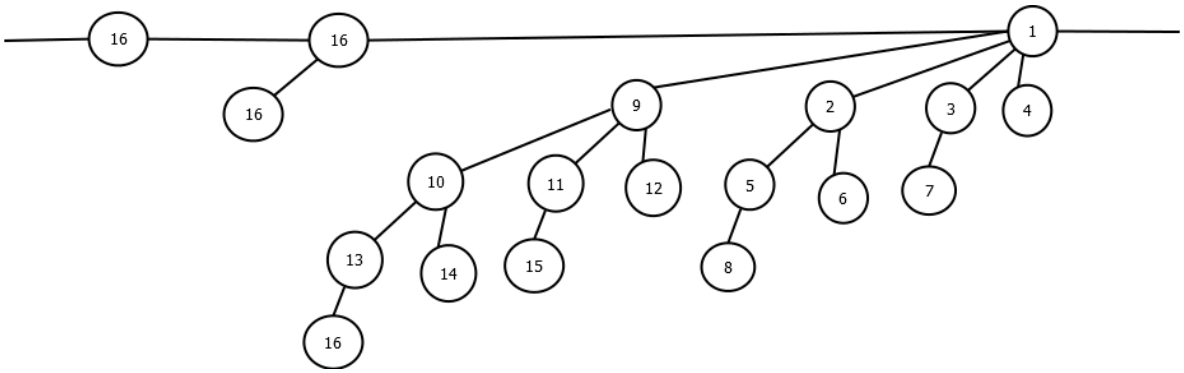


Figure 2. Binomial heap, Fibonacci heap.

Further example of a tournament tree we refer to Figure 3 in the chapter of tournament trees.

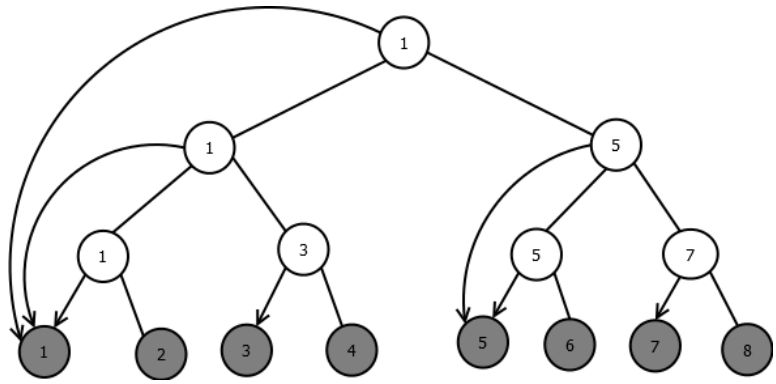


Figure 3. Tournament tree. Leaves are nodes of interest, containing keys. Internal nodes are winners of pairwise tournaments between their two children.

Motivation

Table 1 shows that the most efficient priority queue is a Fibonacci heap, as it provides O(1) amortized time for `Insert, Merge, GetMin,` and `DecreaseKey`, and O(log n) amortized time for `DeleteMin` and `Delete` operations.

Although Fibonacci heaps are efficient from a theoretical point of view, there are some caveats and drawbacks:

- The worst-case performance for `DecreaseKey` might come to $\Theta(n)$[6].
- There are four pointers in total for each node: parent, left, right, and child (which points to the first child). Therefore, it uses more memory than a binary heap.
- The time efficiency of the different operations is based on amortized analysis. A particular call to `ExtractMin`, for example, might take a long time because it's doing leftover work from other operations. With amortized analysis, this additional work is accounted for in the operations that cause the work (J. Boyer[9]).

Some modifications or extensions of Fibonacci heaps that allow slight improvements in structure and algorithms and worth nothing.

Fredman[10] has introduced pairing heaps, which are a self-adjusting variant of Fibonacci heaps. The structure there requires only three or fewer pointers per node, but all operations run in O(log n) amortized time.

Several papers also propose improvements on Fibonacci heaps that keep `DeleteMin` and `Delete` operations in O(log n) amortized time, while the remaining operations take O(1) amortized time.

Diab Abuaiadh and Jeffrey H. Kingston[11] modify several operations done on the Fibonacci heap structure: the consolidation process, where all equal size trees are paired is done in the `GetMin` operation, instead of `DeleteMin`, and the cascading cut is done in both `DecreaseKey` and `Delete` operations. So `DeleteMin` is not presented there as a separate algorithm, but is implemented as a combination of `GetMin` followed by `Delete`. Therefore, the amortized time performance of operations is O(log n) for `GetMin` and `Delete`, and O(1) for the remaining operations. Assuming $t \leq n$, the total time complexity of a sequence of n `Insert`, n `Delete`, m `DecreaseKey` and t `GetMin` operations is $O(n \log n + t \log n + m) = O(n \log n + m)$.

Amr Elmasry[12] introduces violation heaps. Parent pointers are removed, and the left pointer from the first child is used as a parent, the root list is single listed, finally each node, instead of handling a Boolean mark, has an integer goodness property. The feature of this structure is a violation of a node that indicates the level of badness of the node's sub-tree. If the violation of a node is zero, then the size of its sub-tree is exponential with respect to node's rank. Here `DecreaseKey` leads to cutting parent sub-trees if they violate a priority queue property, recalculating the goodness property, or until the new goodness value is less than the old one. `DeleteMin` supposes the cleaning phase that follows actual delete of the node. This is done by repeatedly unlinking the first children of each root until the condition of keeping the first two children of each root without violation is maintained or until the root loses all its children. The cleaning should be done repeatedly for the trees resulting from the unlinking. Using these rules, the author proves that the amortized time for DeleteMin and Delete is O(log n) and O(1) for the remaining operations, the same lower bounds as Fibonacci heaps, while keeping the simpler structure.

G. S. Brodal, G. Lagogiannis, and R. E. Tarjan[13] introduced Strict Fibonacci heaps. This practical implementation of Fibonacci heaps uses the active-passive mark for nodes, to keep the structure of trees and a heap balanced. However, this complicates the structure of the heap: such nodes contain 7 pointers and 2 integers. There are also such heap properties as Active record, Heap record, Rank-list record, and Fix-list record. All of them have a purpose to improve the worst-case efficiency of operations DecreaseKey, DeleteMin, and Merge.

H. Kaplan and R. E. Tarjan[14] discuss other modifications of practical implementations of Fibonacci heaps: thin and thick heaps. Those provide the same amortized efficiency as Fibonacci heaps, but need less space and promise better practical efficiency. The difference between thin and thick heaps is that for the thin heap a node with k children has rank k or k+1, and for the thick heap rank is k or k-1. The modification in the thin or thick heap is that it allows cut only of the first child, so the DecreaseKey that causes the cascading cut might cut only the first children of parents. Between such cuts rank reduction occurs and the sequence may end with a restructuring step where the child becomes a sibling of its parent.

G. S. Brodal[15] offers an implementation of a priority queue that claims to achieve a worst-case time of O(log n) for DeleteMin and Delete, and O(1) for the remaining operations. The data structure in his paper originates from the ideas of the relaxed heaps of Driscoll et al.[16] with the extension of this to allow $\Theta(n)$ heap order violations to support Merge operation in a worst-case constant time and a non-constant number of violations. This heap has a structure similar to Fibonacci heaps.

P. van Emde Boas[17] offers an efficient way of preserving a data structure, based on a binary tree with an average and worst case processing time of O(log log n) per instruction.

Chapter 2. Tournament Trees

Description

Tournament trees are complete binary trees in which the leaves are associated with n elements, each having a key. In addition, there are no nodes with one child, and therefore, there are precisely n-1 internal nodes. Each internal node holds a pointer, σ, to the leaf with the smallest key in its subtree. Leaves point to themselves. For simplicity, the complete binary tree is implemented as an array

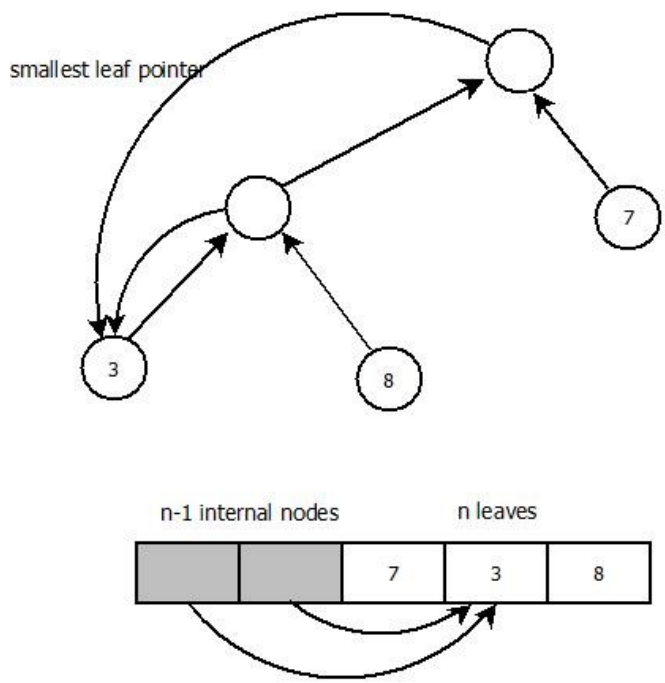


Figure 4. The tournament tree is a complete binary tree in which the leaf nodes correspond to data items with a key. Each internal node has a pointer to the leaf node in its subtree with the smallest key.

Let the nodes be numbered from 1 through 2n – 1. The root is at position 1, and the first n - 1 array entries correspond to internal nodes. The leaves are at positions n through 2n—1. Recall that parent[i] is just $\lfloor i/2 \rfloor$, leftchild[i] is [2i], rightchild[i] is 2i + 1, and a sibling[i] is $i + (-1)^i$ for i > 1.

The tournament tree is particularly well-suited for dealing with priority queue operations such as `Insert` and `DeleteMin`. Its main competition comes from the classical binary heap. However, in just about every possible metric, tournament tree outperforms Binary heaps. The only drawback is that twice as much storage is required, since binary heaps, which are also complete binary trees, store keys at the internal nodes as well.

Operations on Tournament Trees

The data structure supports

(i) `MakeTree.` This creates an array of size 2n - 1 with n keys in the last n slots. It constructs the tournament tree by filling in the pointers **σ**. `MakeTree` can be used to create an empty tree. `MakeTree` takes precisely n - 1 key comparisons, as in a sport tournament.

(ii) `Insert`. Insert a new key.

(iii) `DeleteMin`. Delete the key of minimal value.

We make good use of the atomic operation `Update(x, i)`, which places key x at leaf i, and fixes the pointers on the path from i to the root.

```
Update(x, i)
Key[i] <- x, σ[i] <- i
While i ≠ 1 do
      j <-sibling[i]
      If key[σ[j]] < key[σ[i]]
            σ[parent[i]] <- σ[j]
      Else
            σ[parent[i]] <- σ[i]
      i <- parent[i]
```

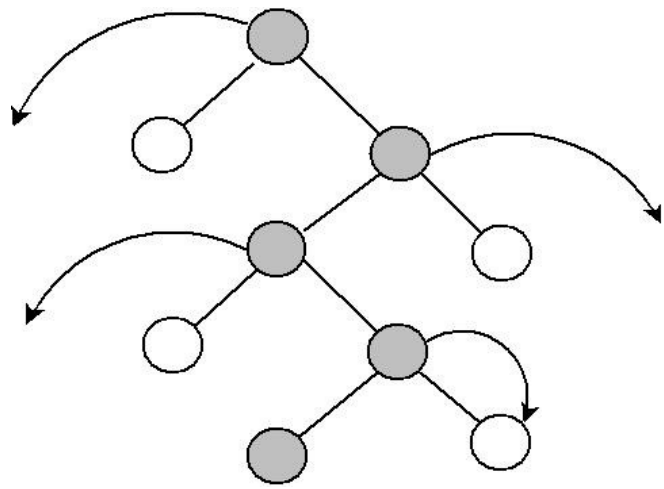


Figure 5. A change in the key value of leaf node i causes an update that affects only the (shaded) path from i to the root of the tournament tree.

If depth[i] $= \lfloor log_2(i) \rfloor$ denotes the depth of node i, then it is easy to see that `Update(x, i)` takes depth[i] $\leq \lfloor log_2(i) \rfloor$ number of element comparisons. Both `Insert` and `DeleteMin` use `Update`.

Insert

```
Insert (x,n) (insert key x into a tree with n elements)
key[2n] <– key[n]
σ[2n] <– 2n, σ[2n + 1] <– 2n + 1
Update (x, 2n + 1)
```

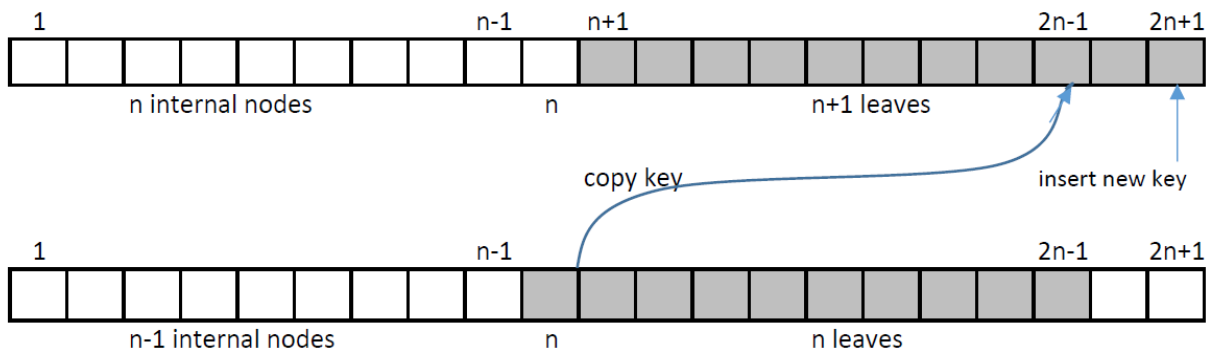


Figure 6. An insertion causes the structure to grow by two nodes. The key at location n is moved to position 2n, and the newly inserted key is placed at position 2n + 1. Additionally, an update started at 2n -1 is required.

DeleteMin

```
DeleteMin() (delete the smallest element of a tree with n + 1 elements)
Report key[1]
Update(∞, σ[1])
```

This is a lazy implementation of `DeleteMin` that creates many keys of value ∞. One can avoid this altogether and make sure that at every point in time, every leaf corresponds to a real key. Assume that the tree has n keys, before `DeleteMin`. Then σ[l] must be replaced by one of the last two keys. We have two candidates. Choosing the one with the larger key has the advantage that the tree can be fixed using just one, not two, calls to `Update`. Here are the details:

```
DeleteMin() (delete the smallest element of a tree with n + 1 elements)
Report key[1]
If σ[1] ≥ 2n
     key[n] <-key[sibling[σ[1]]], σ[1] <- n
Else
     a <- σ[n], b <- sibling[a]
Key[n] <- key[a], σ[n] <- n – 1
Update(key[b], σ[1])
```

The schema of `DeleteMin` operation is shown on Figure 7.

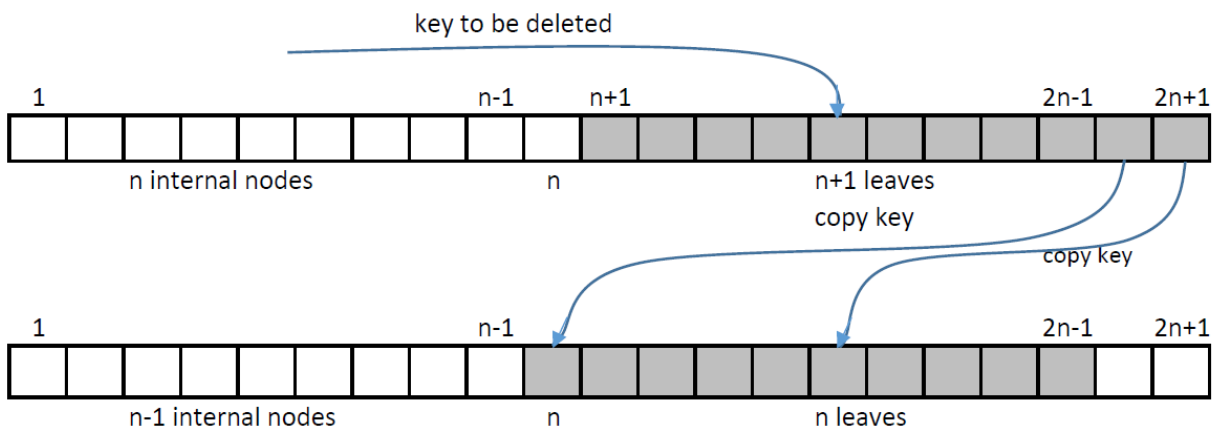


Figure 7. In the operation DeleteMin, the node to be deleted sees its key replaced by one of the two keys in positions 2n and 2n+1. The other key is placed in position n. The figure depicts only one of two possible situations, namely that in which the key at position 2n+1, i.e., σ[n]=2n

MakeTree and Tournament Sort

```
MakeTree(n) (creates a tournament tree with n elements)

Create an array of size 2n – 1 for σ
For i = n to 2n – 1 do σ[i] <– i, set key[i]
For i = n – 1 to 1 do
      If key[σ[2i]] < key[σ[2i + 1]]
            σ[i] <– σ[2i]
      Else
            σ[i] <– σ[2i + 1]
```

After `MakeTree` is done, n consecutive applications of `DeleteMin` yields a sorted list. Since `DeleteMin` uses no more than $\lfloor \log2(2n - 1) \rfloor$ key comparisons, the overall number of comparisons does not exceed

$$n - 1 + \sum_{i=1}^{n} \lfloor \log_2(2i - 1) \rfloor < n + \sum_{i=1}^{n} \log_2(2i) = 2n + \sum_{i=1}^{n} \log_2(i) = 2n + \log_2(n!)$$

This is quite remarkable, as it is within $2n$ comparisons of the theoretical lower bound for sorting, which is $\lceil \log_2(n!) \rceil$. It performs at par with `mergesort`, which requires only $n \log_2 n = \lceil \log_2(n!) \rceil + O(n)$ key comparisons. On the other hand, `heapsort`, which uses a binary heap but is otherwise similar to tournament sort, requires about $2n \log_2 n$ key comparisons in the worst case.

Chapter 3. Tournament Heaps

Introduction

Our new data structure, called the **tournament heap**, is based on the tournament tree. It supports all priority queue operations listed in the introduction, matching Fibonacci heaps for amortized complexities.

Recall that a tournament simulates a tennis tournament. We say that a leaf node is a player. Players can play games at internal nodes, leaving a winner and a loser. All players beaten by a player are called the children of that player.

Eventually, from a set of players, we end up with one winner, so it becomes a root of the tournament tree. In the tournament heap we have many sets of players with their tournaments, where there were no games between players of different sets. Therefore, we combine them into a heap, where we define the "grand" winner, which becomes a tournament heap minimum, called Min.

Structure

The tournament heap has the following structure:

1. **Root list**. This is a linked list consisting of nodes. This is a single-linked list.
2. **Last**. This is a last node of a root list.
3. Each node of a root list is a root of a tournament tree, if there were no `DecreaseKey` or `DeleteMin` operations.
4. Nodes of each tournament tree can be of type **leaf** and type **internal**. Leaf node is an actual player. Internal node is a "representation" of the leaf, which is a winner in the last tournament.
5. **Children** are leaves that are beaten by current node as a player. As inner node represents a leaf (player), by the child of a current node we represent the topmost inner node of a corresponding leaf.
6. **Successors** are two nodes that have a ParentNext pointer (defined below) to the current node.
7. **Min**. This is a leaf with min value of all keys in the entire tournament heap.

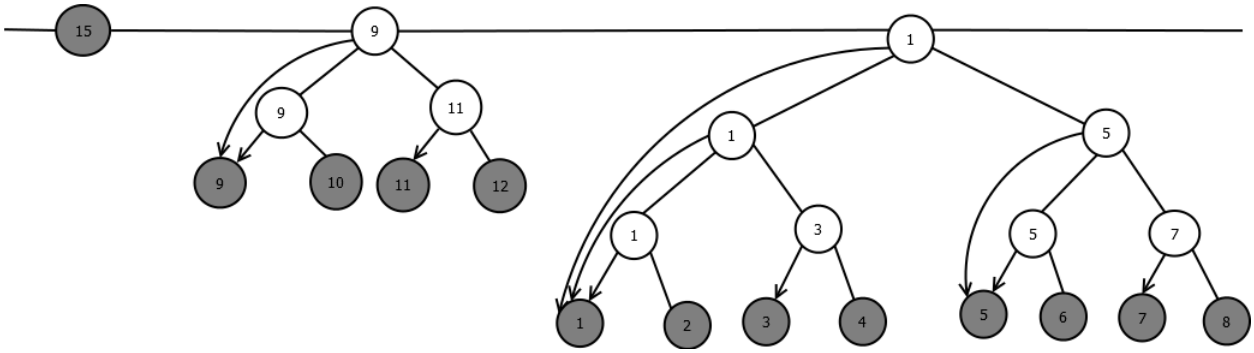


Figure 8. Tournament heap.

Nodes of a tournament heap have the following structure.

Internal and leaf nodes

- **ParentNext** pointer. This pointer points to the parent internal node, or to the next root list node, in case if current node is a root of the tree.

Internal node

- **Winner** pointer (σ in tournament tree). Winner points to the leaf node that contains the smallest key in its sub-tree.
- **Loser** pointer. For an internal node points to one of its successors, i.e. the one that corresponds to the loser.
- **WinnerSuccessor** pointer. This pointer points to the non-loser successor (see above for the loser definition). For leaves this pointer is null, as they do not have successors.

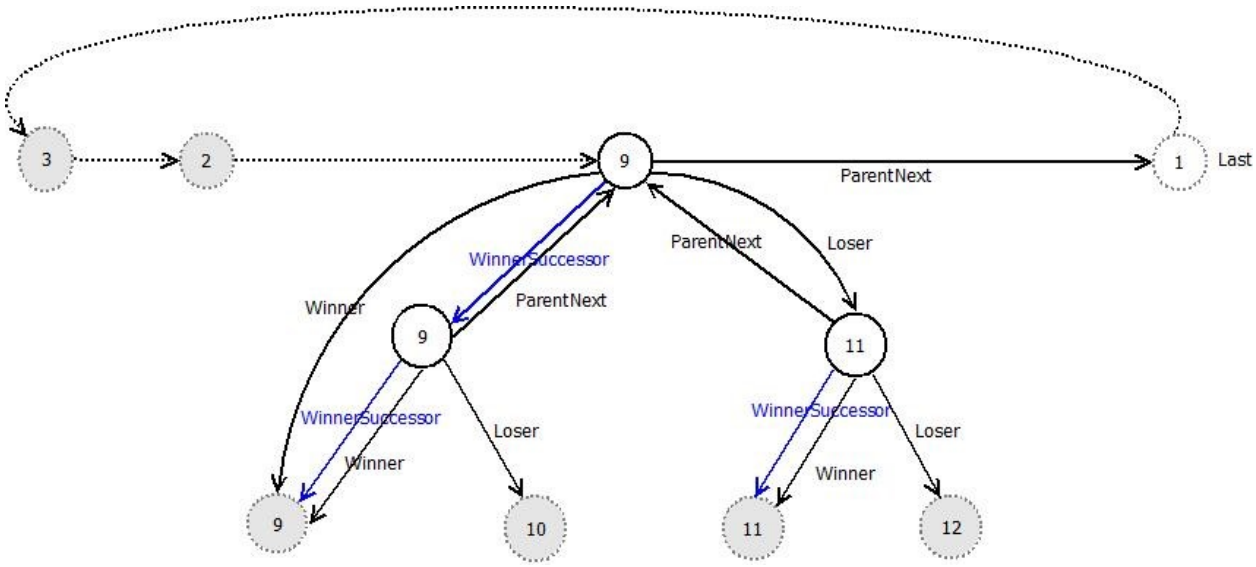


Figure 9. Internal nodes properties structure. Leaves are shaded.

Leaf node

- **Winner** pointer. Winner points to the highest internal node, which points to that leaf.
- **Key**. This is a property of comparable type, such as integer, float, or double. This is the target property of the leaf node for which all comparisons are.

- **Mark**. This is a property of Boolean type. Mark is set to true in case when a player has lost one of its children.
- **Rank**. This is a property of integer type that shows the number of children of a node.

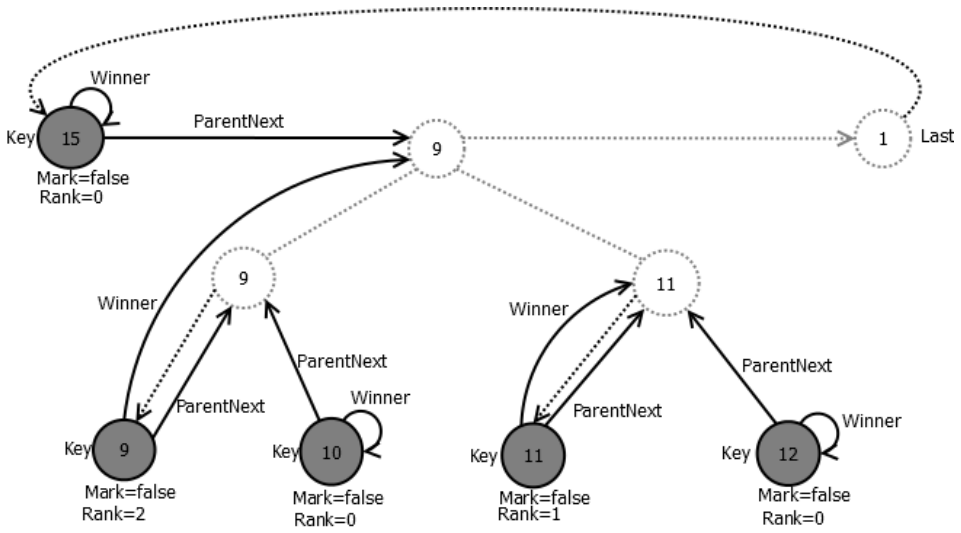


Figure 10. Leaf nodes properties structure.

The table showing all internal and leaf nodes properties is shown below on Table 2.

Table 2. Tournament heap: internal and leaf nodes structure.

Node type	Property	Type	Notes
Internal Node	ParentNext	pointer	Parent for non-root, Next root for root
	WinnerSuccessor	pointer	Points to winner successor.
	Winner	pointer	Points to a leaf with min key in node's subtree
	Loser	pointer	Points to loser successor.
Leaf Node	ParentNext	pointer	Parent for non-root, Next root for root
	Winner	pointer	Points to highest winner internal node.
	Key	comparable	
	Mark	Boolean	
	Rank	Integer	

Rules

There are some rules that a tournament heap must follow:

1. Each internal node should have exactly two successors.
2. If a leaf is marked “true”, then no internal node between the leaf and the leaf’s Winner can disappear.
3. The root cannot lose a child. Instead, we will introduce the “swap” operation further on.

Operations

In this section we represent the descriptions of all operations on tournament heaps.

The root list is a single-linked list. When there is only one node it points to itself with ParentNext.

As the root list becomes circular, we define the **Last** node of the root list. The **Last** shows the “end” of the root list, and new nodes are added after the **Last** and they become **Last**.

We suppose that initially an empty structure of the tournament heap is created during the construction of the object. In other words, when we create new tournament heap, an empty root list is created, and Min equals to null.

There is an atomic operation used for `Insert`, `DeleteMin`, `DecreaseKey`, and `Delete` operations called `InsertNodeToRootlist`. It adds a node into a root list and reconnect ParentNext pointers.

We also consider that the internal node is a “representation” of a leaf, as it shows the place where the leaf would be after a tournament. So we would use the leaf properties for all nodes, where it is necessary.

We use the abbreviation PN for ParentNext pointer.

As a naming conventions, we use `x, y, z` for nodes, and `key` for integer keys.

```
InsertNodeToRootlist(x)
x.mark <- false
If root list is empty
      add x as a single root list element
      Min = x
      Last = x
Else
      Insert x after the Last element of a root list and make it Last
      Min <- minimum(Min.key, x.key)
```

Insert

Here we create a new node from a new value into an empty or non-empty Root list.

```
Insert(k)
x <- new node(key <- k)
InsertNodeToRootlist(x)
```

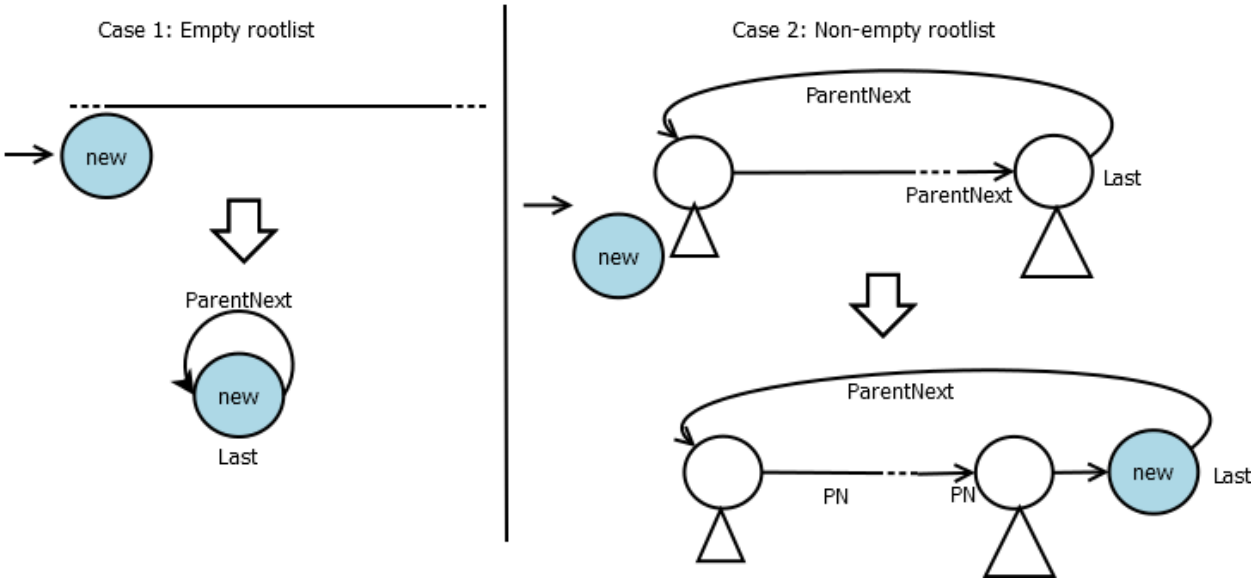


Figure 11. Insert new key operation.

Merge

```
Merge(h1, h2)
Min <- Minimum(h1.Min, h2.Min) // Min of the new merged heap.
Insert the root list of h2 after the h1.Last
```

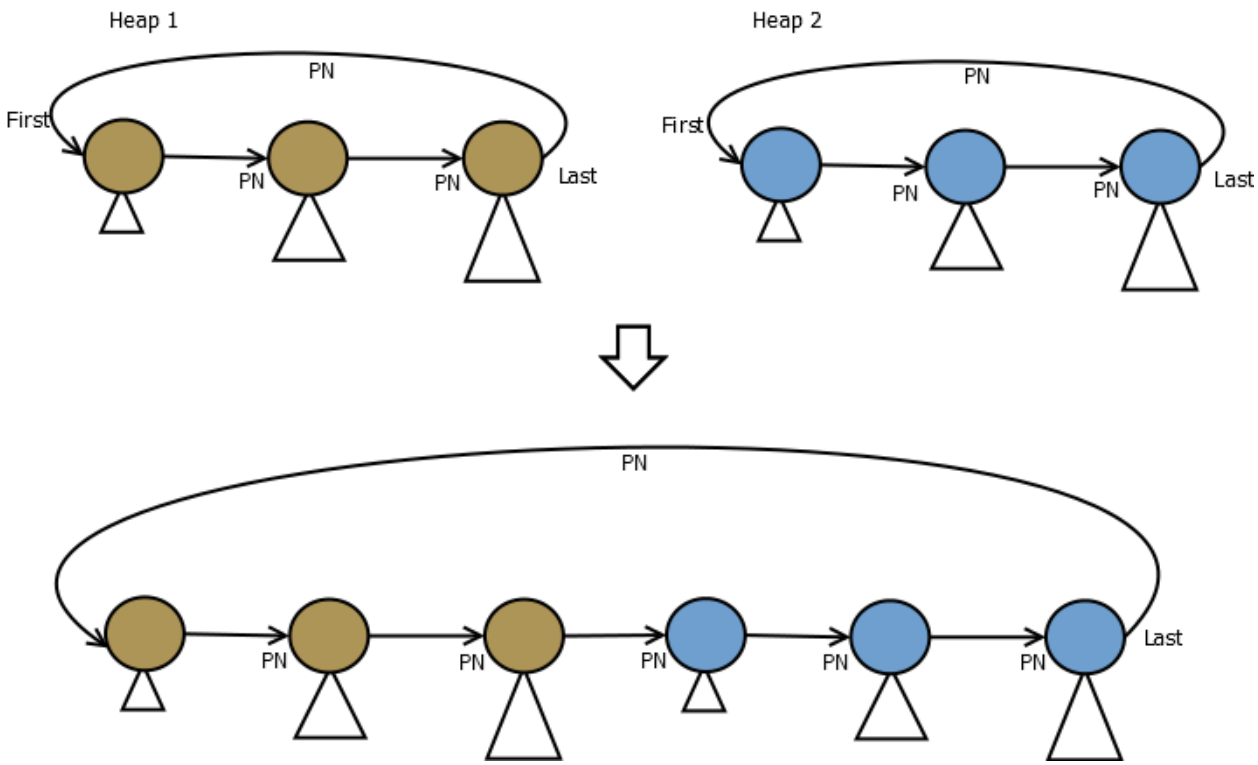


Figure 12. Merge operation.

DeleteMin

During this operation we consolidate the data, similar as for Fibonacci heaps. Place all children of the Min player in the root list at a cost equal to the number of such players. Then for all root list trees of the same rank, this operation keep linking pairwise those until all root list trees are of different sizes using the `Consolidate` operation.

```
DeleteMin()
x <- Min
AddNodeParentsToRootlist(x) //x.Winner is a highest winner of the min,
which is a root
Consolidate()
return x
```

When the minimal leaf has been removed, on the way from that leaf up to the root, parent's remaining children with their subtrees must be moved to the root list using the recursive function.

```
AddNodeParentsToRootlist(x)
If x is on root list
     return
Else
     parent <- x.parentNext
     loser <- parent.loser
     InsertNodeToRootlist(loser)
     AddNodeParentsToRootlist(parent)
```

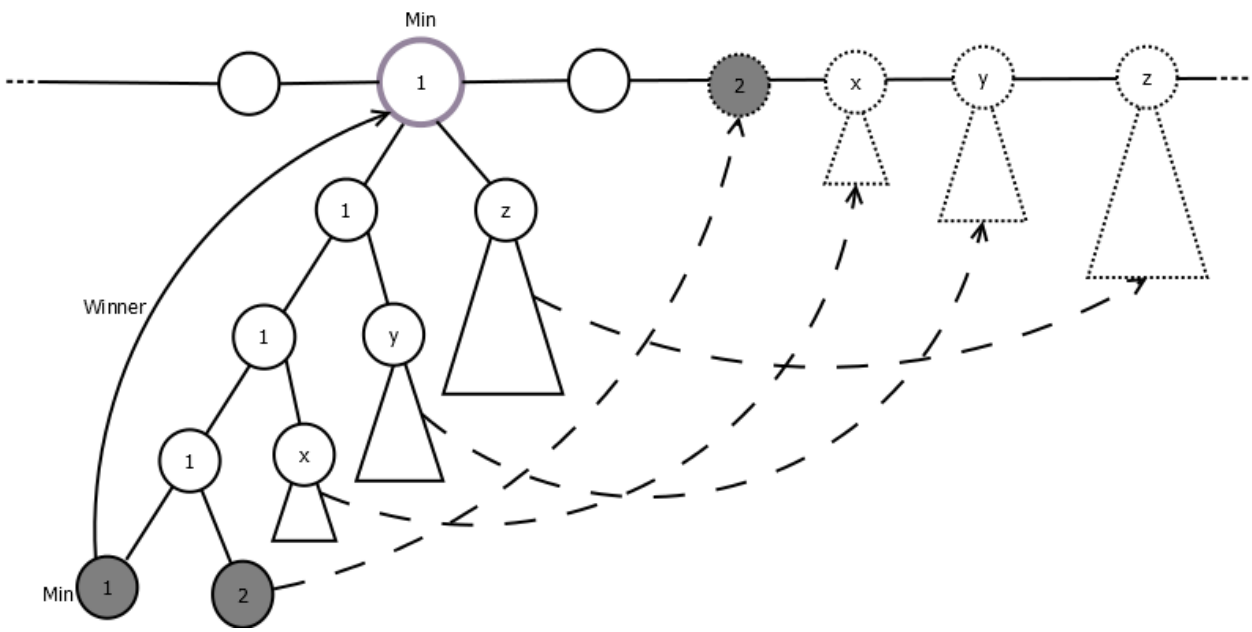


Figure 13. DeleteMin operation.

The `Consolidation` step is similar to that in the Fibonacci heap, except that the `Link` operation creates a tournament tree.

```
Consolidate()
Let ranks[0…maxRank] be a new empty array of nodes, where min rank is
the maximal rank encountered.
First <- Last.ParentNext
For each node x in the root list, starting from First
      If x = Min //as Min is deleted element we skip it.
            Continue
      Else
            z <- x
            r <- z.rank
            while ranks[r] ≠ nil
                  y <- ranks[r] //existing node with the same rank
                  z <- Link(z, y)
```

```
                ranks[r] <- nil
                r++
            ranks[r] <- z

Create new empty root list // Withdraw the old root list with its
deleted Min
For i = 0 to maxRank
     If ranks[i] ≠ nil
          InsertNodeToRootlist(ranks[i]) //this takes care of
assigning Min also
```

The `Link` operation creates a new parent root with two equal rank children. One is a Winner, and one a Loser. `Link` does not insert this new tree into a root list.

```
Link(x1, x2)
Create new internal node p
win <- minimum(x1, x2), los <- maximum(x1, x2)
winnerLeaf <- win.winner
p.winner <- winnerLeaf
p.winnerSuccessor <- win
p.loser <- los
x1.parentNext = x2.parentNext <- p
winnerLeaf.winner <- p
return p
```

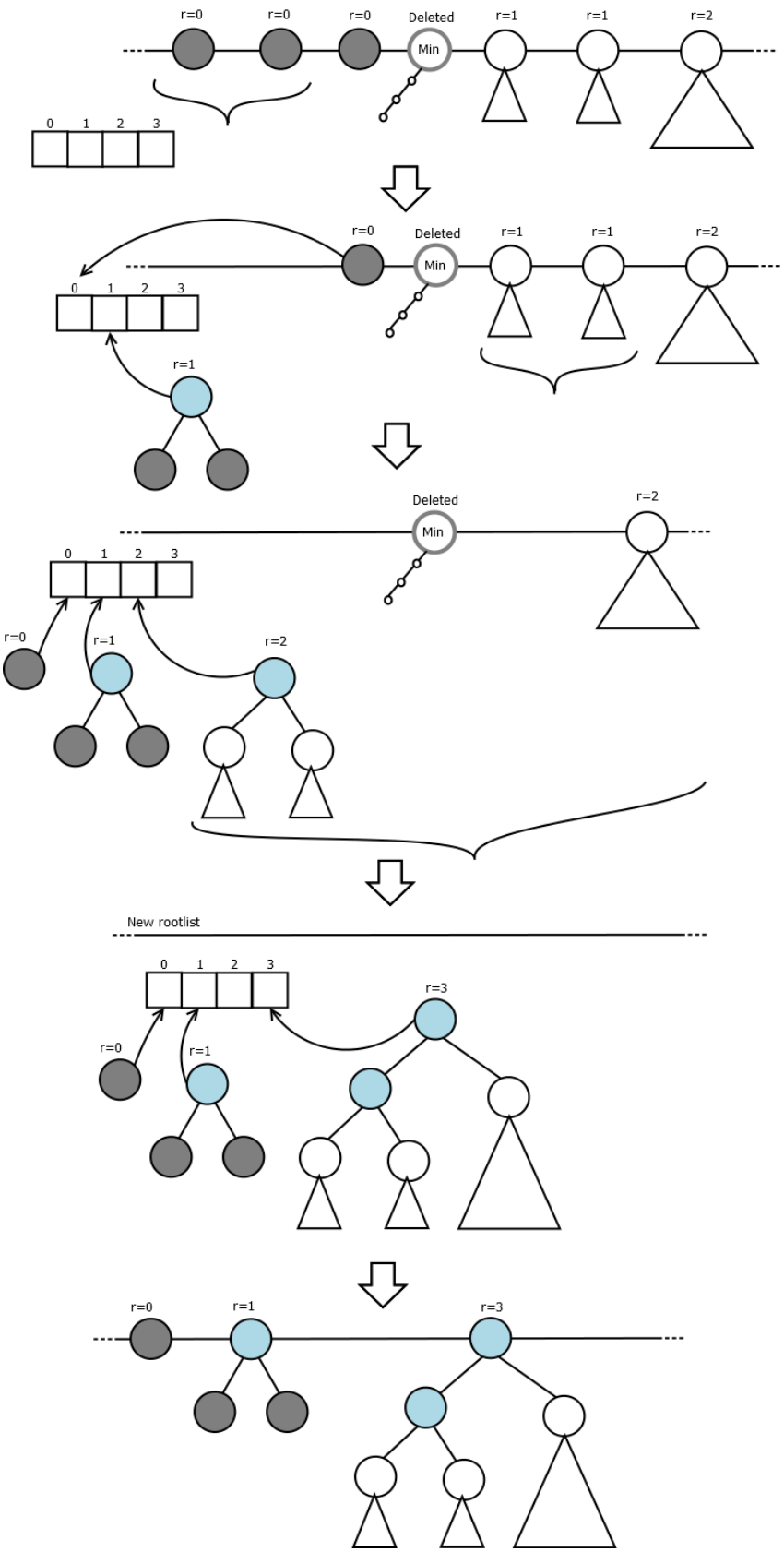


Figure 14. DeleteMin - Consolidation. The light blue nodes are created in a Link operation.

DecreaseKey

The `DecreaseKey` decreases a key and checks if the tournament tree property is violated or not. If yes, then sub-trees where the violation happens are cut and inserted into the root list. Auxiliary operations are `SwapKeys`, `Cut`, and `CascadingCut`.

```
DecreaseKey(leaf, k)
   leaf.key <- k
   If leaf is root
      update Min if needed and return
   Else (if leaf is not root)
      parent <- leaf.parentNext
      //case 1
      If key of parent.winner.key ≤ leaf.key
            return //if parent's leaf hasn't changed
      Else If parent.winner.key > leaf.key
            highestWinner <- parent.winner.winner
            //case 2
            If highestWinner is root
                  SwapKeys(parent.winner, leaf)
                  update Min if needed and return
            //case 3
            Else //leaf.key < non-root highest winner
                  SwapKeys(parent.winner, leaf)
                  CascadingCut(highestWinner)


SwapKeys(leaf1, leaf2)
      Swaps leaf1.key and leaf2.key
```

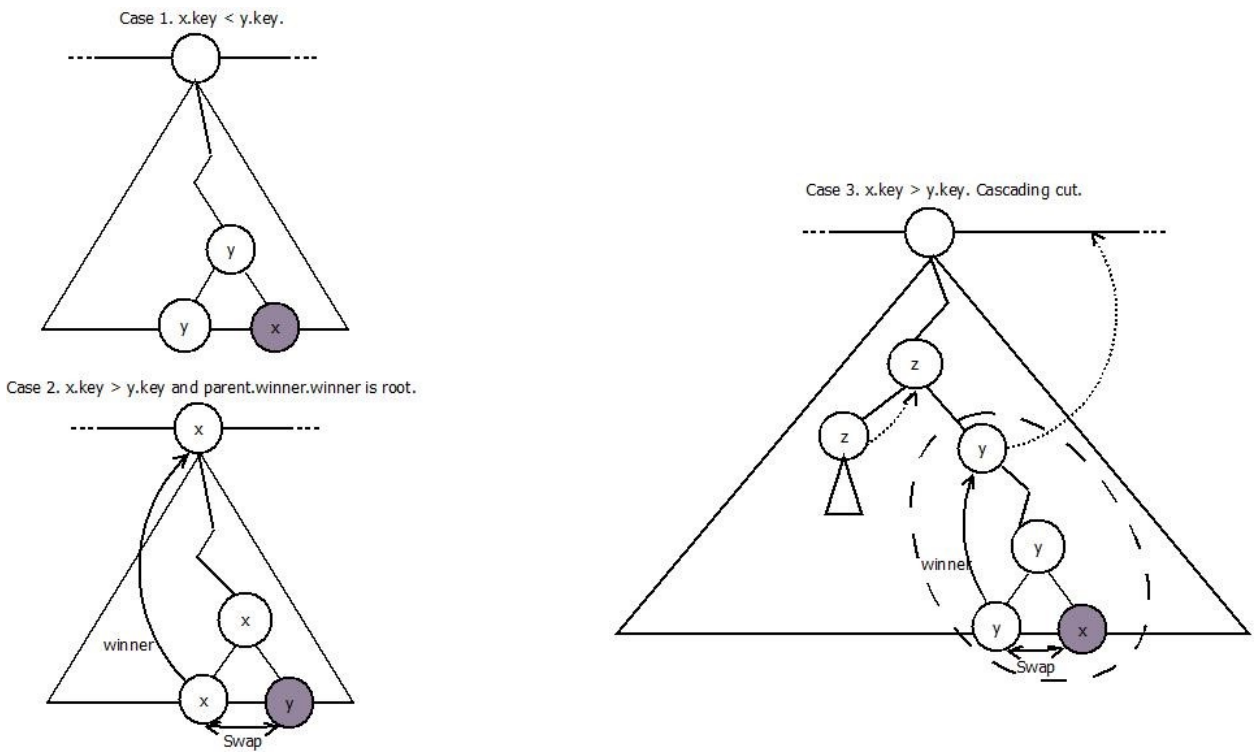


Figure 15. DecreaseKey operation. Shaded is the node with decreased key.

The `CascadingCut` operation cuts the subtree of the target node only in case if node's parent's winner leaf is marked as false and the resulting new value of the node is less than its parent's winner. In this case, to keep the priority queue structure not violated, we do cut of the node, and mark the parent's winner leaf as true. In case if that parent's leaf is marked as true, then we cannot cut the node, but instead we go to the leaf's highest winner node to do a CascadingCut operation recursively with the last one.

```
CascadingCut(internalNode)
     parent <- internalNode.ParentNext
      winnerSibling <- parent.winnerSuccessor
      leaf <- parent.winner
      If leaf.Mark = false
            put internalNode to root list
            leaf.Mark = true
            leaf.Rank--
       Else If leaf.Mark = true
```

```
SwapKeys(leaf.winner, internalNode.wineer)
CascadingCut(leaf.winner) //highest winner of a leaf
```

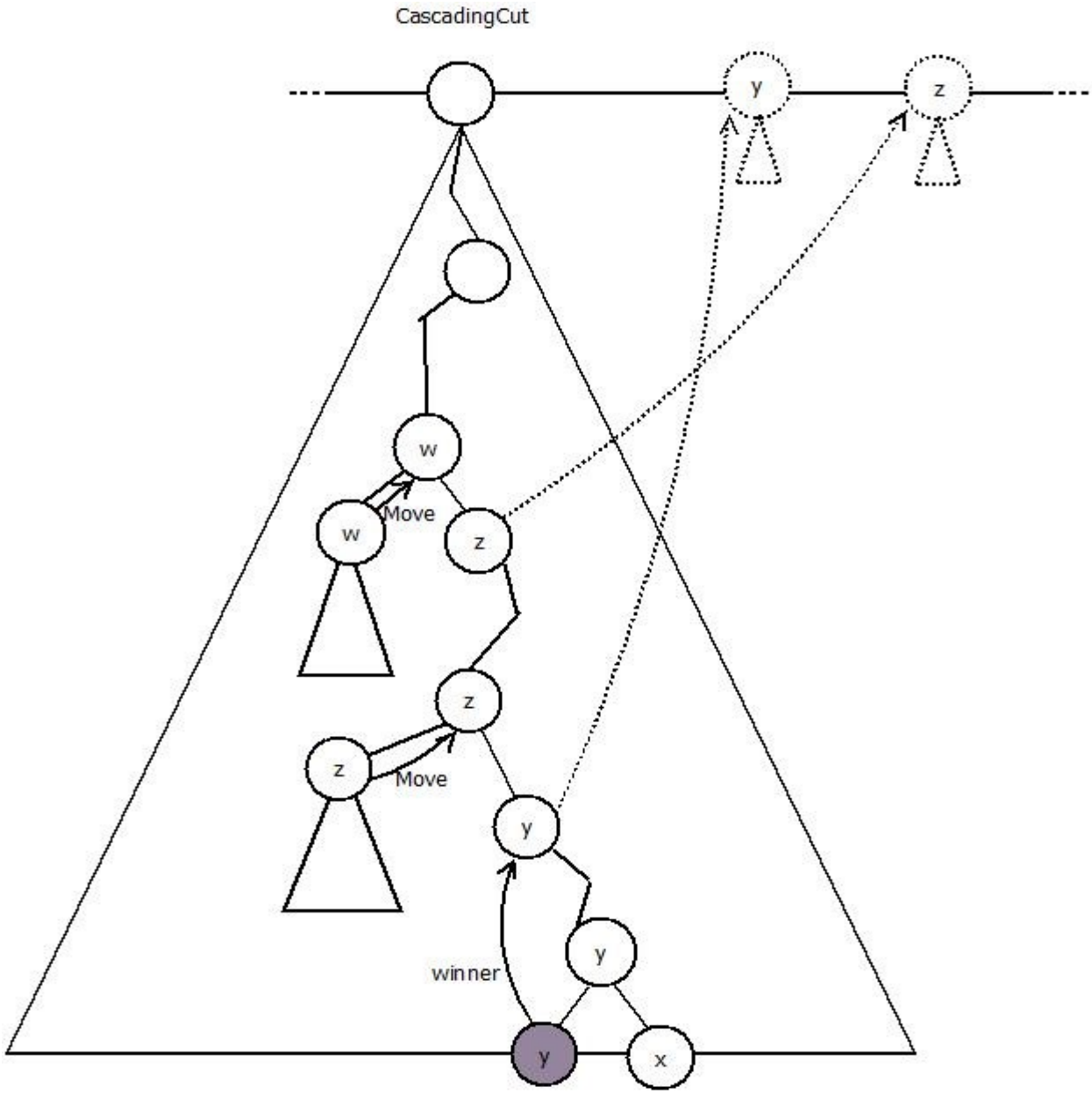


Figure 16. DecreaseKey – CascadingCut.

Delete

The `Delete` operation can be represented as a combination of `DecreaseKey` when decreasing the key to -∞, followed by `DeleteMin` operation.

Chapter 4. Analysis

For efficiency, the tournament heap should be in a balanced state at every moment. So for the tournament heap with n elements following conditions should met.

1. The number of trees after consolidation should not exceed $O(\log_2 n)$.
2. For every tree in the heap, with the rank **r**, the number of leaves should satisfy:
   $\#\text{leaves} \geq A\, C^r$, (1)
   where **A** and **C** are some constants.

Ranks and Tree Sizes

When equal rank root list trees merge, one of them increases its rank by one.

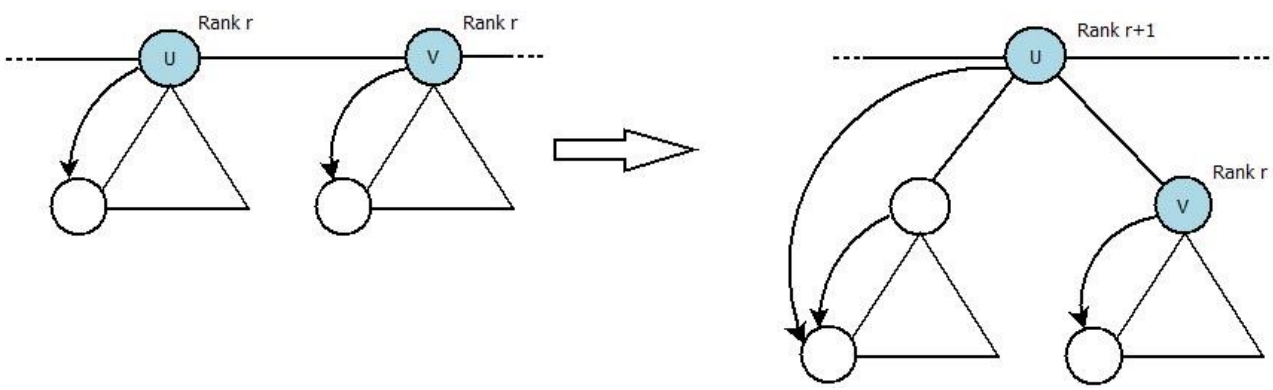


Figure 17. Link equal rank trees. Winner root increases rank by 1

According to rule 2, the rank of a node in a tree can decrease by at most one. Thus, if the root of a tree has rank r, then its children have the original ranks that they had at the time they joined the tree or one less. In particular, the highest child of the root has rank at least r-2, the second highest one – rank at least r-3, and so on.

Let T_r be the minimal possible size of a tree whose root has a rank r, where the size is measured by the number of leaves (or number of players).

We have $T_0 = 1$ and $T_1 = 2$ since the root cannot lose its child. Furthermore:

$$T_r \geq T_{r-2} + T_{r-3} + \cdots + T_2 + T_1 + T_0 + T_0$$

This is explained by considering two cases. If the root has a rank r at link time, then, as a root, it cannot lose a child, by rule 3, and thus its rank remains unchanged. All of its children can decrease their ranks by at most one, thus explaining inequality. If the root becomes a child, then it can decrease its rank possibly by one, and in that case, the inequality is most certainly satisfied.

We show that $T_r \geq C\,\varphi^r$, where $\varphi = \frac{1+\sqrt{5}}{2}$ is the golden ratio, i. e., the largest root of $x^2 = x + 1$. To see this, we use induction on r. We will see that $C = \varphi - 1$ will do.

For r = 0, 1, we have

$1 = T_0 \geq C\,\varphi^0 = C$

$2 = T_1 \geq C\,\varphi^1 = C\,\frac{1+\sqrt{5}}{2}$, provided that $C \leq 1$.

Assume now $r \geq 2$. Then we verify the inequality as follows:

$T_r \geq T_{r-2} + T_{r-3} + \cdots + T_2 + T_1 + T_0 + T_0 \geq C\,(\varphi^{r-2} + \varphi^{r-3} + \cdots + \varphi^0) + 1$ (by hypothesis)

$= C\,\varphi^{r-2}\left(1 + \frac{1}{\varphi} + \cdots + \frac{1}{\varphi^{r-2}}\right) + 1 = C\,\frac{\varphi^{r-1}-1}{\varphi-1} \geq C\varphi^r$,

where the last inequality holds when

$C\,\varphi^{r-1} - C + \varphi - 1 \geq C\,\varphi^{r+1} - C\,\varphi^r$,

i. e., since

$\varphi^{r+1} = \varphi^r + \varphi^{r-1}$, when $C \leq \varphi - 1$.

So we can take $C = \varphi - 1$, and we are done.

As a consequence, since

$n \geq \max_r T_r \geq C \max_r \varphi^r = C\,\varphi^{\max_r r}$,

we see that

$\max r \leq \frac{\log(\frac{n}{C})}{\log \varphi}$.

Thus the maximal rank is $O(\log n)$

Verification of the amortized analysis claims

We take the potential function

$\Phi = \alpha$ (# of trees in root list) + β (# marked nodes).

A marked node is a non-root node, which has lost one child and is defined by the mark in a leaf node on which the current node's winner points. When a node becomes a root, it is unmarked and treated as if it has never lost a child.

Insert

With this choice of Φ, and as it does not include loops or recursions, it is clear that `Insert` takes amortized and actual time O(1).

Merge

Merge is an O(1) time operation.

DecreaseKey

The `DecreaseKey` the operation by itself increases Φ by at most $\alpha + \beta$, and thus has an amortized and an actual time of O(1). However, it may start a `CascadingCut` operation. One step in `CascadingCut` operation takes a constant actual time of O(1). In addition, if it is not the final step in the `CascadingCut` (so that a node is placed on the root list), Φ decreases by β (one less marked node) and increases by α (one more tree in the root list). When $\beta = \alpha + 1$, the amortized cost of a non-final step is zero. Only the final step in a `CascadingCut` contributes to the amortized time by a value equal to 1.

DeleteMin

Finally consider `DeleteMin`. All k trees in the root list are placed in a bin by rank of their roots, for a cost of k. These bins are processed by merging until no bin has two equal rank trees. Each merge step takes time O(1) and decreases Φ by α. Thus such a step has an amortized cost of 1 – α.

We end up with $l \leq \frac{\log n}{\lfloor \log \varphi \rfloor} + 1$ trees, since the largest rank is less than or equal to $\frac{\log n}{\lfloor \log \varphi \rfloor}$.

Thus the total amortized cost of `DeleteMin` does not exceed

$k + (k - l)(1 - \alpha) = k(2 - \alpha) - l(1 - \alpha) = l$ (if we set $\alpha = 2$) $\leq O(\log n)$.

Delete

Similarly `Delete` has an O(log n) amortized cost. Since α = 2, we can choose β = 3, thus concluding the amortized analysis.

Chapter 5. Comparison with Fibonacci heap

The tournament heap is constructed having tournament trees as its components, and the operational basis of Fibonacci heap. Therefore, it has similarities and differences with Fibonacci heaps.

Similarities

1. The heap structure. Both heaps consist of the circular root list containing roots of trees they composed from.
2. Both root lists have a Min property.
3. Similar node properties:
    a. Parent. In the tournament heap, ParentNext plays a parent role.
    b. Child. They refer to the first direct child.
4. Operations work in the same time: O(1) actual time for `Insert`, `Merge`, `GetMin`, O(1) amortized time for `DecreaseKey`, and O(log n) for `DeleteMin` and `Delete`.
5. Operations Insert, Merge, and GetMin are similar in implementation.
6. Operation DeleteMin is similar for both heaps, except that the Consolidation creates different types of trees.

Differences

The differences between the tournament heap and Fibonacci heap are represented in the table 3.

Table 3. Comparison of tournament heap and Fibonacci heap

	Tournament heap	**Fibonacci heap**
Tree type	Binary. Only two successors (direct children).	Binomial. Unlimited number of direct children.
Node type	Two types: leaf and internal node. Leaf contains information of Key, Mark, and Rank, as well as pointers. Internal node contains only pointers.	All nodes are of one type. Each contains Key, Mark, Rank and required pointers.
Rank	Leaf's property, showing how many children it has.	Node's property, showing the height of the node's subtree. Usually it is equal to the number of direct children.
Pointers per node	4 for internal node. 2 for leaf.	4
Root list	Single-linked.	Doubly-linked.
Marking rule	Leaf is marked when one of internal nodes on the path from that leaf to the root is losing the child.	Node is marked when one child of that node is cut.
Marking consequence	No more than one node cut may happen in the path from the marked leaf to the root.	No more than two children cuts may happen for each node.
CascadingCut reason	Leaf key is less than non-root highest winner.	Second child cut.
Potential function	Φ = #roots + #marked nodes	Φ = # roots + 2 x #marked nodes

Simulation

In order to verify the efficiency of the tournament heap, we made a series of experiments. Implementations of tournament heaps and Fibonacci heaps were done in C#.

The programming code was thoroughly tested in the Microsoft Unit Testing project, which permits automatic verification of predefined conditions.

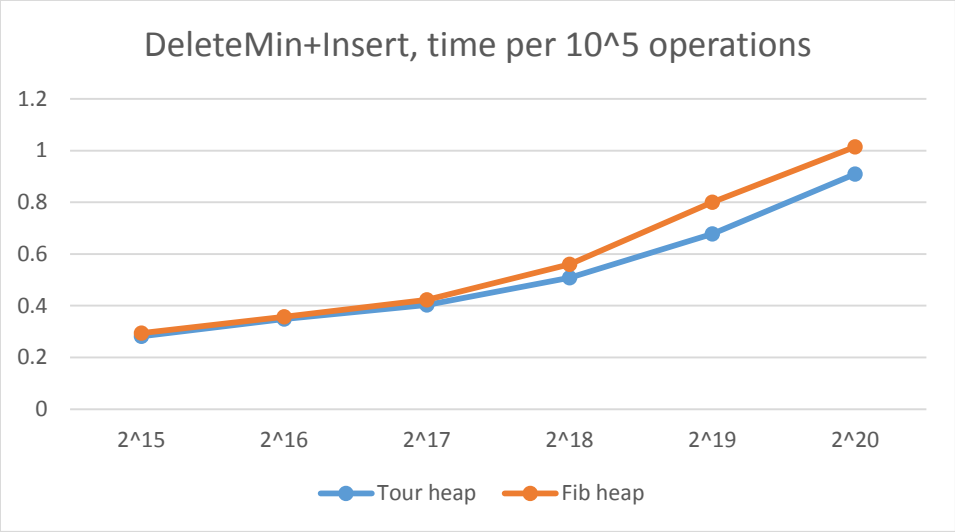
For both tournament and Fibonacci heaps we were using the same sets of data varying in size from 2^{15} to 2^{20}. For each data set **x**, we were doing a number of operations DecreaseKey and DeleteMin followed by Insert, varying from 10^5 to 5 x 10^6. For each data set and each number of operations there were 5 to 10 experiments, from which the minimal, maximal, and average times were recorded for both heaps. All records are shown in table 4.

Table 4. Actual time results of long run of operations on tournament heap and Fibonacci heap.

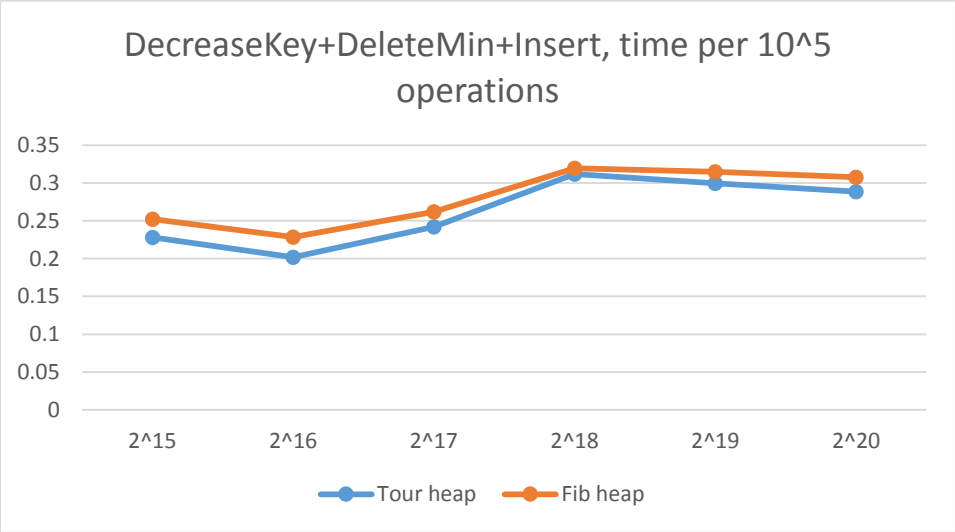
# of nodes to Insert (+ one DeleteMin)	**Operations**	**# of operations**	**Actual Time Tournament Heap, per 10^5**			**Actual Time Fibonacci heap, per 10^5**		
			Min	**Max**	**Average**	**Min**	**Max**	**Average**
2^20	Inserts	5*10^6	0.03102199	0.0330021	0.0320353	0.0227613	0.0264413	0.024788
	DeleteMin + Insert	5*10^6	0.89686213	0.9166021	0.9094918	0.9882423	1.0556828	1.0139161
	DecreaseKey + DeleteMin + Insert	5*10^6	0.28033804	0.2962746	0.2887774	0.3009638	0.3153372	0.30763
	10 DecreaseKey + DeleteMin + Insert	10^6	0.4906989	0.8984978	0.684175	0.5672595	0.8984978	0.7969475
	100 DecreaseKey + DeleteMin + Insert	10^5	3.0783187	3.1567419	3.1149045	4.5161229	4.6567879	4.5577345
	100 DeleteMin+ 100 Insert	10^5	64.3816896	71.178079	67.686704	115.4096	117.18866	115.93277
2^19	Inserts	5*10^6	0.01338077	0.0159409	0.0140307	0.0125407	0.0173613	0.0145777
	DeleteMin + Insert	5*10^6	0.65221726	0.7005799	0.6776721	0.7943656	0.8057256	0.8006056

	DecreaseKey + DeleteMin + Insert	5*10^6	0.24937428	0.3626411	0.2996141	0.2638157	0.370641	0.3146181
	10 DecreaseKey + DeleteMin + Insert	10^6	0.49402648	0.7049403	0.6019174	0.5624344	0.8279476	0.7061406
	100 DecreaseKey + DeleteMin + Insert	10^5	3.0691579	3.4022097	3.2071879	4.3582693	4.6092494	4.4877553
	100 DeleteMin + 100 Insert	10^5	54.5441094	59.8883 92	57.3487 69	80.2005 75	81.1126 4	80.6584 31
2^18	Inserts	5*10^6	0.00488071	0.0051003	0.0049904	0.0058203	0.0074804	0.0062104
	DeleteMin + Insert	5*10^6	0.47642757	0.5306899	0.5081157	0.5555717	0.5650119	0.5602487
	DecreaseKey + DeleteMin + Insert	5*10^6	0.25587424	0.3618606	0.3117009	0.2701149	0.3689408	0.3194546
	10 DecreaseKey + DeleteMin + Insert	10^6	0.48752952	0.5818315	0.5323979	0.5897312	0.6983399	0.6288341
	100 DecreaseKey + DeleteMin + Insert	10^5	2.7631563	3.0771945	2.9150058	3.894218	4.6202598	4.2444052
	100 DeleteMin + 100 Insert	10^5	45.9616263	48.209766	47.237525	60.231457	61.009452	60.658292
2^17	Inserts	5*10^6	0.00221982	0.0038602	0.0030635	0.0016601	0.0045003	0.0029769
	DeleteMin + Insert	5*10^6	0.38552198	0.4134436	0.4033698	0.4169239	0.4278644	0.4229441
	DecreaseKey + DeleteMin + Insert	5*10^6	0.21279169	0.2828755	0.2419235	0.2393735	0.2938565	0.2620346
	10 DecreaseKey + DeleteMin + Insert	10^6	0.41512068	0.5181262	0.469493	0.5034284	0.5977361	0.5547312
	100 DecreaseKey + DeleteMin + Insert	10^5	2.3881399	2.5971463	2.4918005	3.4112093	4.3142391	3.7787139
	100 DeleteMin + 100 Insert	10^5	41.676365	43.15146	42.350748	48.363769	49.691827	48.943958
2^16	Inserts	5*10^6	0.00101972	0.0019801	0.0013567	0.00042	0.0027001	0.0015901
	DeleteMin + Insert	5*10^6	0.34849991	0.35006	0.3490465	0.3547602	0.3581806	0.3567736
	DecreaseKey + DeleteMin + Insert	5*10^6	0.16680983	0.2388132	0.2016461	0.197971	0.257523	0.2285709
	10 DecreaseKey + DeleteMin + Insert	10^6	0.32082042	0.3717199	0.3493868	0.3166201	0.4128251	0.367172
	100 DecreaseKey + DeleteMin + Insert	10^5	1.4300778	1.8391053	1.5320901	1.9081069	2.1441182	2.039448
	100 DeleteMin + 100 Insert	10^5	39.0262107	39.86429	39.191246	42.423415	42.842441	42.650269
2^15	Inserts	5*10^6	0.00022002	0.0006603	0.0004102	0.0004197	0.0010601	0.0007267
	DeleteMin + Insert	5*10^6	0.27545566	0.2864963	0.2822263	0.2886162	0.2985371	0.2940133
	DecreaseKey + DeleteMin + Insert	5*10^6	0.18935045	0.3043774	0.2282195	0.2155923	0.3170985	0.2520078
	10 DecreaseKey + DeleteMin + Insert	10^6	0.3378168	0.4526234	0.3729038	0.3471226	0.4451278	0.3851563
	100 DecreaseKey + DeleteMin + Insert	10^5	1.4970867	1.7580983	1.620085	2.0931219	2.2861509	2.2106343
	100 DeleteMin + 100 Insert	10^5	35.6090308	36.993128	36.211398	35.799028	37.046095	36.376066

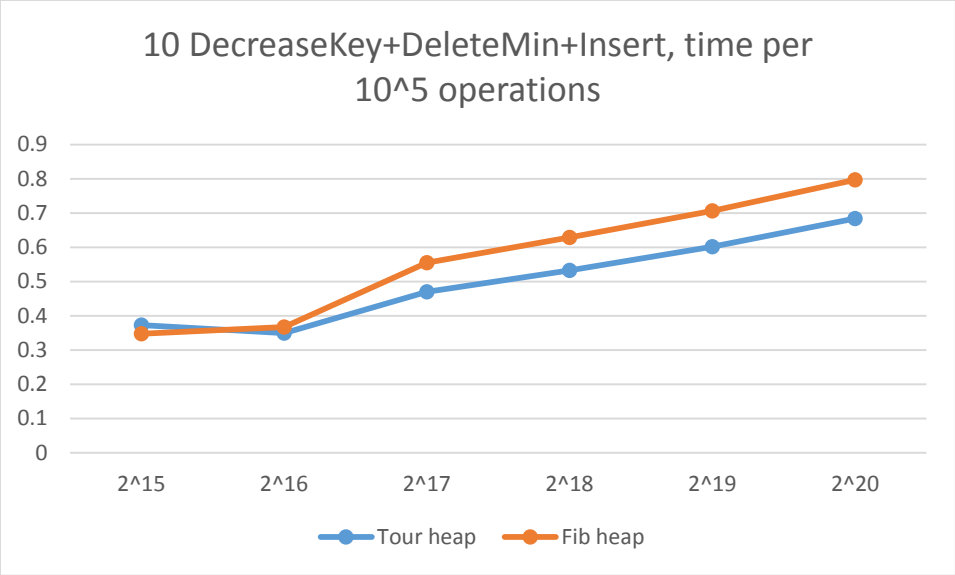
Results of time to complete, per 100,000 operations for all four operations sets for both tournament and Fibonacci heaps are plotted on graphs represented below.



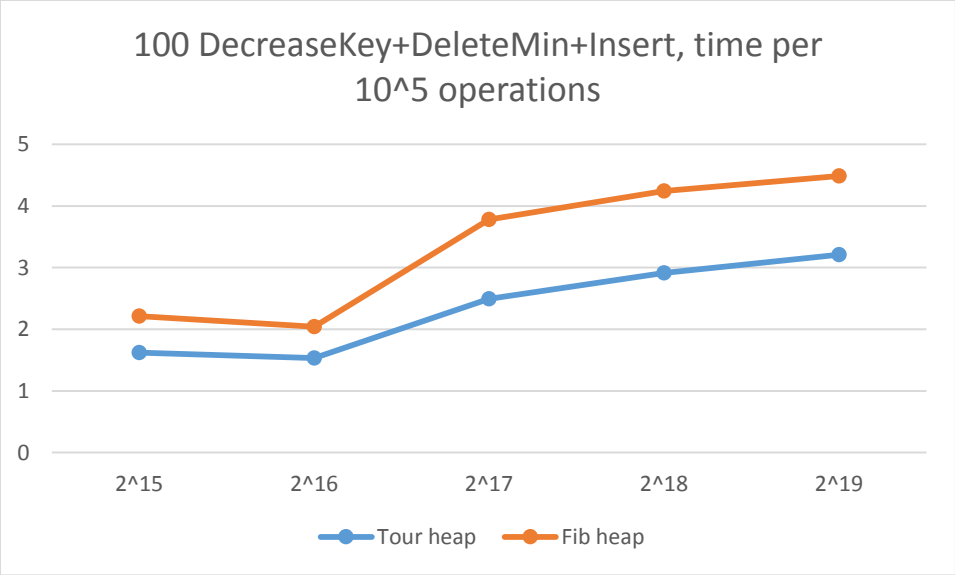
Graph 1. Comparison graph of DeleteMin + Insert operations.



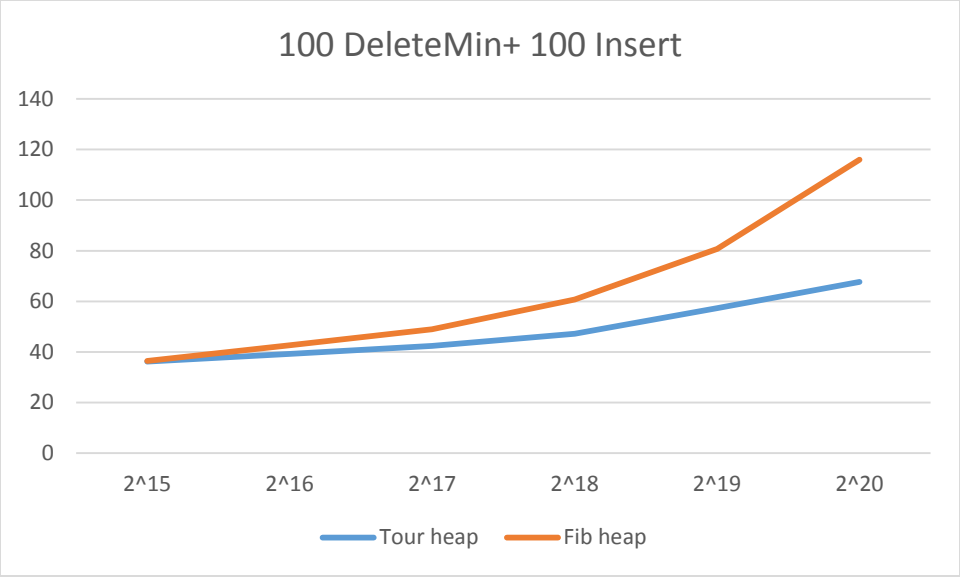
Graph 2. Comparison graph of DecreaseKey + DeleteMin + Insert operations.



Graph 3. Comparison graph of 10 DecreaseKey + DeleteMin + Insert operations.



Graph 4. Comparison graph of 100 DecreaseKey + DeleteMin + Insert operations.



Graph 5. . Comparison graph of 100 DeleteMin + 100 Insert operations.

Conclusion

In this thesis, we provided an alternative to the Fibonacci heap that is at the same time simpler, easier to implement and theoretically and experimentally at least as good as the Fibonacci heap.

This new structure, the tournament heap, could serve as a replacement of the popular Fibonacci heap if speed is at a premium.

References

1. M. T. Goodrich and R. Tamassia, "9.3. Heaps" in *Data Structures and Algorithms in Java,* 6th ed, John Wiley & Sons, 2014.
2. T. Cormen, *et al*., *Introduction to Algorithms*, 3rd Edition, Cambridge, Massachusetts, The MIT Press, 2009.
3. R. E. Tarjan, "Data Structures and Network Algorithms", Society for Industrial and Applied Mathematics, 1983.
4. Chris L. Kuszmaul. (2009, November, 16). Dictionary of Algorithms and Data Structures, ed., U.S. National Institute of Standards and Technology [Online]. Available: *http://xlinux.nist.gov/dads//HTML/binaryheap.html*.
5. J.W.J. Williams, "Algorithm 232: Heapsort", in *Communications of the ACM*, Vol. 7, pp 347-348, 1964.
6. Jean Vuillemin, "A data structure for manipulating priority queues", in *Communications of the ACM*, Volume 21, #4, pp. 309-315, 1978.
7. M. Fredman and R. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms", in *ACM Transactions on Algorithms*, Vol. 34, pp. 598-605, 1987.
8. L. Devroye, "Tournament Trees" in *Comp 610, Information Systems*, School of Computer Science. McGill University. 2014.
9. J. Boyer, "Algorithm Alley: Resizable Data Structures", in Dr. Dobb's Journal, 23(1), pp. 115-116, January 1998.
10. M. Fredman, "On the efficiency of pairing heaps and related data structures", in *ACM Transactions on Algorithms,* Vol. 46, No 4, pp. 473-501, 1999.
11. D. Abuaiadh and J. H. Kingston, "Are Fibonacci Heaps Optimal?", Basser Dept. of Computer Science, The University of Sydney, Australia, 2006.
12. A. Elmasry, "Violation Heaps: A Better Substitute for Fibonacci Heaps", Max-Planck Institut fur Informatik Saarbrucken, Germany.

13. G. S. Brodal, et al., “Strict Fibonacci Heaps”, Dept. of Computer Science, Aarhus University, 2012.
14. H. Kaplan and R. E. Tarjan, “Thin Heaps, Thick Heaps”, in *ACM Transactions on Algorithms*, Vol. 4, No 1, pp. 1-14, 2008.
15. G. S. Brodal, “Worst-Case Efficient Priority Queues”, in *Proceedings of the seventh annual ACM-SIAM symposium on Discrete algorithms*, Society for Industrial and Applied Mathematics, pp 52-58, 1996.
16. J. R. Driscoll, et al., “Relaxed heaps: An alternative to Fibonacci heaps with applications to parallel computation”, in Communications of the ACM, Vol. 31, pp 1343-1354, 1988.
17. P. van Emde Boas, “*Preserving order in a forest in less than logarithmic time”*. Proc. IEEE, pp. 75-84. IEEE Computer Society, 1975.

Appendix A
Listing of programming code

The practical implementation of the tournament heap and Fibonacci heap, for comparison, is done using Microsoft Visual Studio 2013, C# programming language.

In the code, only primitive data types, such as Integer, String, Boolean, and Array, and operations, such as +, -, * were used in order to avoid unnecessary hidden language operations that could add an extra time to the process.

Code also contains testing methods that help to visualize trees.

Tournament heap

```
    public class TournamentHeap
    {
        #region Attributes
        private TournamentElement[] _elements;
        private int _numberOfElements;
        private List<int> _deletedElementIndices = new List<int>();
        #endregion

        #region Properties
        public TournamentElement[] Elements
        {
            get { return _elements ?? (_elements = new TournamentElement[NumberOfElements
* 100]); }
            set { _elements = value; }
        }
        public int LastElementIndex { get; set; }
        public TournamentNode Last { get; set; }
        public Leaf Min { get; set; }
        public int TotalNum { get; set; }
        public int MaxDegree { get; set; }
        public int NumberOfElements
        {
            get
            {
                if (_numberOfElements == 0)
                {
                    _numberOfElements = 1000;
                }
                return _numberOfElements;
            }
            set { _numberOfElements = value; }
        }
        #endregion

        #region Methods
        #region Private Methods
```

```
        public Leaf NodeLeaf(TournamentNode node) { return (node.IsLeaf) ? (Leaf)node :
((InternalNode)node).Winner; }

        private void CheckMin(Leaf leaf){ if (Min == null || leaf.Key < Min.Key) { Min =
leaf; } }
        private void InsertNodeToRootlist(TournamentNode newRoot, bool checkMin = true)
        {
            var leaf = NodeLeaf(newRoot);
            leaf.Mark = false;
            newRoot.IsRoot = true;
            if (Last == null) //new empty root list
            {
                Last = newRoot;
                Last.ParentNext = Last;
                Min = leaf;
            }
            else
            {
                var first = Last.ParentNext;
                Last.ParentNext = newRoot;
                newRoot.ParentNext = first;
                Last = newRoot;
                if (checkMin){ CheckMin(leaf); }
            }
        }
        #endregion
        public void Insert(int newKey)
        {
            var leafNode = new Leaf { Key = newKey };//Rank=0, Mark=false, winner=null
            var element = new TournamentElement {Node = leafNode};
            leafNode.Element = element;
            if (_deletedElementIndices.Any())
            {
                int index = _deletedElementIndices.First();
                element.Index = index;
                Elements[index] = element;
                _deletedElementIndices.RemoveAt(0);
            }
            else
            {
                element.Index = ++LastElementIndex;
                Elements[LastElementIndex] = element;
            }
            InsertNodeToRootlist(leafNode);
            TotalNum++;
        }
        public static TournamentHeap Merge(TournamentHeap heap1, TournamentHeap heap2)
        {
            if (heap2.Last == null) return heap1;
            if (heap1.Last == null) return heap2;

            heap1.CheckMin(heap2.Min);
            var heap1First = heap1.Last.ParentNext;
            var heap2First = heap2.Last.ParentNext;
            heap1.Last.ParentNext = heap2First;
            heap2.Last.ParentNext = heap1First;
            heap1.TotalNum += heap2.TotalNum;
            return heap1;
```

```
        }
        public Leaf DeleteMin()
        {
            if (Min == null) return null;
            var removedMin = Min;
            if (Min.Rank > 0) //Min is not a root node. Min is to be actually removed in
Consolidate()
            { AddNodeParentsToRootlist(Min); }
            TotalNum--;
            _deletedElementIndices.Add(Min.Element.Index);
            Elements[Min.Element.Index] = null;
            Consolidate();
            return removedMin;
        }
        #region Private methods for DeleteMin
        /// <param name="node">node along the path to root</param>
        /// <param name="endNote">node until which, exclusive, recursion should
go</param>
        private void AddNodeParentsToRootlist(TournamentNode node)
         {
            if (node.IsRoot) return;

            var parent = (InternalNode)node.ParentNext;
            InsertNodeToRootlist(parent.Loser, false);//loser always exists
            AddNodeParentsToRootlist(parent);
        }
        private void Consolidate()
        {
            var maxRank = (int) Math.Log(TotalNum, 2)+1;
            var arrayRanks = new TournamentNode[maxRank + 10];
            var first = Last.ParentNext;
            var currentRoot = first;
            do
            {
                var currentRootLeaf = NodeLeaf(currentRoot);
                var nextCurrent = currentRoot.ParentNext;
                if (currentRootLeaf != null && currentRootLeaf != Min)//after cascading
cut can appear an empty internal node
                {
                    var rank = currentRootLeaf.Rank;
                    while (arrayRanks[rank] != null)
                    {
                        if (MaxDegree < rank + 1) { MaxDegree = rank + 1; }
                        currentRoot = Link(currentRoot, arrayRanks[rank]);
                        arrayRanks[rank] = null;
                        rank++;
                    }
                    arrayRanks[rank] = currentRoot;
                }//as Min is deleted element we skip it
                currentRoot = nextCurrent;
            } while (currentRoot != first);

            Min = null; Last = null;
            foreach (var currRoot in arrayRanks.Where(currRoot => currRoot != null))
            {
                InsertNodeToRootlist(currRoot);
            }
        }
```

```
        private TournamentNode Link(TournamentNode first, TournamentNode second)
        {
            var parent = new InternalNode();
            first.IsRoot = second.IsRoot = false;
            var firstLeaf = NodeLeaf(first);
            var secondLeaf = NodeLeaf(second);
            var firstIsWinner = firstLeaf.Key < secondLeaf.Key;
            var winnerLeaf = (firstIsWinner) ? firstLeaf : secondLeaf;
            parent.WinnerSuccessor = (firstIsWinner) ? first : second;
            parent.Loser = (firstIsWinner) ? second : first;
            parent.Winner = winnerLeaf;
            first.ParentNext = second.ParentNext = winnerLeaf.Winner = parent;
            winnerLeaf.Rank++;
            return parent;
        }
        #endregion

        public void DecreaseKey(Leaf leafToDecrease, int decreasedVal)
        {
            leafToDecrease.Key -= decreasedVal;
            var newKey = leafToDecrease.Key;
            if (leafToDecrease.IsRoot) { CheckMin(leafToDecrease); return; }//leaf on the
root list

            //leaf not on the root list
            var parent = (InternalNode)leafToDecrease.ParentNext;
            if (leafToDecrease != parent.Winner)//if leafToDecrease is loser, then
parent.Winner is its sibling
            {
                if (parent.Winner.Key <= newKey) { return; }//case1: parent's leaf hasn't
changed
                SwapKeys(leafToDecrease, parent.Winner);
                leafToDecrease = parent.Winner;
            }

            var highestWinner = parent.Winner.Winner;
            if (highestWinner.IsRoot)
            {
                if (leafToDecrease != parent.Winner) { SwapKeys(parent.Winner,
leafToDecrease); }//leaf is not tree min
                CheckMin(parent.Winner); //case 2: leaf's parent on the way from tree min
to root
            }
            else if (((InternalNode) highestWinner.ParentNext).Winner.Key > newKey)
            {
                 SwapKeys(leafToDecrease, parent.Winner);
                 CascadingCut(highestWinner);
            }//case 3
        }
        #region Private methods for DecreaseKey
        private void SwapKeys(Leaf leaf1, Leaf leaf2)
        {
            int temp = leaf1.Key;
            leaf1.Key = leaf2.Key;
            leaf2.Key = temp;
        }
        private void CascadingCut(InternalNode node)
        {
```

```
            if (node.IsRoot) return;

            var parent = (InternalNode)node.ParentNext;
            var winnerSibling = parent.WinnerSuccessor;
            var leaf = parent.Winner;
            if (leaf.Mark)//highest winner of a leaf
            {
                SwapKeys(node.Winner, leaf);
                CascadingCut(leaf.Winner);
            }
            else
            {
                InsertNodeToRootlist(node);
                leaf.Rank--;
                if (parent.IsRoot)
                {
                    if (winnerSibling.IsLeaf)
                    {
                        InsertNodeToRootlist(winnerSibling);
                        parent. WinnerSuccessor = parent.Loser = parent.Winner = null;
                    }
                    else
                    {
                        parent. WinnerSuccessor = ((InternalNode)winnerSibling).
WinnerSuccessor;
                        parent.Loser = ((InternalNode) winnerSibling).Loser;
                        ((InternalNode)winnerSibling). WinnerSuccessor.ParentNext =
parent;
                        ((InternalNode) winnerSibling).Loser.ParentNext = parent;
                    }
                }
                else//put remaining node into parent's place, as otherwise parent has
only one successor: contradicts tournament
                {
                    leaf.Mark = true;
                    var grandParent = (InternalNode)parent.ParentNext;
                    if (parent == grandParent. WinnerSuccessor)
                    {
                        grandParent.WinnerSuccessor = winnerSibling;
                    }
                    else
                    {
                        grandParent.Loser = winnerSibling;
                        if (leaf.Winner == parent && leaf != winnerSibling)
                        {
                            leaf.Winner = (InternalNode)winnerSibling;
                        }
                    }
                    winnerSibling.ParentNext = grandParent;
                }
            }
        }
        #endregion
        #endregion

        #region Test methods
        public string GetRootlistKeys()
        {
```

```
            var sb = new StringBuilder();
            var sbRanks = new StringBuilder();
            sb.AppendFormat("Root list: ");
            sbRanks.AppendFormat("Ranks: ");
            if (Last == null)
            {
                sb.Append("Empty heap");
            }
            else
            {
                var first = Last.ParentNext;
                sb.AppendFormat("{0}", NodeLeaf(first).Key);
                sbRanks.AppendFormat("{0}", NodeLeaf(first).Rank);
                var current = first.ParentNext;
                while (current != first)
                {
                    var notRoot = (current.IsRoot) ? string.Empty : " nr";
                    sb.AppendFormat("-{0}{1}", NodeLeaf(current).Key, notRoot);
                    sbRanks.AppendFormat("-{0}", NodeLeaf(current).Rank);
                    current = current.ParentNext;
                }
            }
            return sb + ";" + sbRanks;
        }
        public string GetTreeLeaves(TournamentNode node, int level = 0)
        {
            if (node == null) return "nill";
            var sb = new StringBuilder();
            sb.AppendFormat("Lv{0}: ", level);
            var nodeLeaf = NodeLeaf(node);
            var mark = (nodeLeaf.Mark) ? "m" : string.Empty;
            if (node.IsLeaf)
            {
                sb.AppendFormat("{0}{1}", nodeLeaf.Key, mark);
            }
            else if (nodeLeaf.ParentNext == node)
            {
                sb.AppendFormat("Lv{0}: ", ++level);
                sb.AppendFormat("Cld{2} {0}{1}", nodeLeaf.Key, mark, level);
                var loser = (Leaf) ((InternalNode) node).Loser;
                if (loser != null)
                {
                    var markl = (loser.Mark) ? "m" : string.Empty;
                    sb.AppendFormat("- Lsr{2} {0}{1}", loser.Key, markl, level);
                }
            }
            else
            {
                var internalNode = (InternalNode) node;
                level++;
                sb.Append(string.Format("Cld{0} {1}", level, GetTreeLeaves(internalNode.
WinnerSuccessor, level)));
                sb.AppendFormat(" | Lsr{0} {1}", level, GetTreeLeaves(internalNode.Loser,
level));
            }

            return sb.ToString();
        }
```

```
private int CalcRoots(TournamentNode node)
{
    if (node == null) return 0;
    var roots = 0;
    if (node.IsRoot) roots++;
    if (!node.IsLeaf)
    {
        var intnode = (InternalNode) node;
        roots += CalcRoots(intnode. WinnerSuccessor);
        roots += CalcRoots(intnode.Loser);
    }
    return roots;
}
public bool CorrectRoots()
{
    var roots1 = 0;
    var roots2 = 0;

    var first = Last.ParentNext;
    if (first.IsRoot)
    {
        roots1++;
    }
    else return false;
    roots2 += CalcRoots(first);

    var current = first.ParentNext;
    while (current != first)
    {
        if (current.IsRoot) roots1++;
        else return false;
        roots2 += CalcRoots(current);
        current = current.ParentNext;
    }
    return roots1 == roots2;
}
#endregion

public class TournamentNode
{
    public TournamentNode ParentNext { get; set; }
    public bool IsLeaf { get; set; }
    public bool IsRoot { get; set; }
}

public class InternalNode : TournamentNode
{
    public Leaf Winner { get; set; }
    public TournamentNode Loser { get; set; }
    public TournamentNode WinnerSuccessor { get; set; }

    public InternalNode() { IsLeaf = false; }
}

public class Leaf : TournamentNode
{
    public InternalNode Winner { get; set; }
```

```
            public int Key { get; set; }
            public bool Mark { get; set; }
            public TournamentElement Element { get; set; }
            public int Rank { get; set; }

            public Leaf(){ IsLeaf = true; }
        }

        public class TournamentElement
        {
            public int Index { get; set; }
            public Leaf Node { get; set; }
        }
    }
```

Fibonacci heap

```
    public class FibonacciHeap
    {
        #region Attributes
        private FibonacciElement[] _elements;
        private int _numberOfElements;
        private List<int> _deletedElementIndices = new List<int>();
        #endregion

        #region Properties
        public int NumberOfElements
        {
            get
            {
                if (_numberOfElements == 0) { _numberOfElements = 1000; }
                return _numberOfElements;
            }
            set { _numberOfElements = value; }
        }
        public FibonacciElement[] Elements
        {
            get { return _elements ?? (_elements = new FibonacciElement[NumberOfElements
* 100]); }
            set { _elements = value; }
        }
        public int LastElementIndex { get; set; }
        public FibonacciNode RootlistFirst { get; set; }

        public FibonacciNode Min { get; set; }
        public int TotalNum { get; set; }
        public int MaxDegree { get; set; }
        #endregion

        #region Methods
        #region Private Methods

        private void CheckMin(FibonacciNode node) { if (Min == null || node.Key <
Min.Key) { Min = node; } }
        private void InsertNodeToRootlist(FibonacciNode node)
```

```
{
    node.Parent = null;
    node.Mark = false;
    if (RootlistFirst == null)
    {
        RootlistFirst = node;
        RootlistFirst.Next = RootlistFirst;
        RootlistFirst.Prev = RootlistFirst;
        Min = RootlistFirst;
    }
    else
    {
        var listLast = RootlistFirst.Prev;
        RootlistFirst.Prev = node;
        node.Next = RootlistFirst;
        listLast.Next = node;
        node.Prev = listLast;
    }
}
#endregion

public void Insert(int newKey)
{
    var element = new FibonacciElement();
    var node = new FibonacciNode{ Element = element, Key = newKey };
    element.Node = node;
    if (_deletedElementIndices.Any())
    {
        int index = _deletedElementIndices.First();
        element.Index = index;
        Elements[index] = element;
        _deletedElementIndices.RemoveAt(0);
    }
    else
    {
        element.Index = ++LastElementIndex;
        Elements[LastElementIndex] = element;
    }
    InsertNodeToRootlist(node);
    CheckMin(node);
    TotalNum++;
}
public void Merge(FibonacciHeap heap2)
{
    CheckMin(heap2.Min);
    var rootlistLast = RootlistFirst.Prev;
    var rootlistLast2 = heap2.RootlistFirst.Prev;
    RootlistFirst.Prev = rootlistLast2;
    rootlistLast2.Next = RootlistFirst;
    rootlistLast.Next = heap2.RootlistFirst;
    heap2.RootlistFirst.Prev = rootlistLast;
    TotalNum += heap2.TotalNum;
}
public FibonacciNode DeleteMin()
{
    if (Min == null) return null;
    var removedMin = Min;
    _deletedElementIndices.Add(Min.Element.Index);
```

```
        Elements[Min.Element.Index] = null;
        TotalNum--;
        //move all children of Min to root list
        if (Min.Child!= null)
        {
            var lastChild = Min.Child.Prev;
            var curChild = Min. Child;
            if (curChild == lastChild) { InsertNodeToRootlist(curChild); }
            else
            {
                var nextChild = curChild.Next;
                do
                {
                    InsertNodeToRootlist(curChild);
                    curChild = nextChild;
                    nextChild = curChild.Next;
                } while (curChild != lastChild);
                InsertNodeToRootlist(lastChild);
            }
            Min. Child = null;
        }
        //remove Min and assign Min to its right child, then consolidate to fix heap
        var minRight = Min.Next;
        if (minRight == Min)
        {
            Min = null;
            if (TotalNum == 0)
            {
                RootlistFirst = null;
            }
        }
        else
        {
            var minLeft = Min.Prev;
            minLeft.Next = minRight;
            minRight.Prev = minLeft;
            Min = minRight;
            if (RootlistFirst == removedMin) { RootlistFirst = minRight; }
            Consolidate();
        }
        //NumNodes--;
        return removedMin;
    }
    #region Private Methods for DeleteMin
    private void DeleteRootFromList(FibonacciNode node)
    {
        if (node == null) return;

        var prev = node.Prev;
        var next = node.Next;
        if (RootlistFirst == node) { RootlistFirst = next; }
        prev.Next = next;
        next.Prev = prev;
    }

    private FibonacciNode Link(FibonacciNode first, FibonacciNode second)
    {
        DeleteRootFromList(second);
```

```
            if (first.Child == null)
            {
                first.Child = second;
                second.Next = second;
                second.Prev = second;
            }
            else
            {
                var listLast = first.Child.Prev;
                var listFirst = first.Child;
                listFirst.Prev = second;
                second.Next = listFirst;
                listLast.Next = second;
                second.Prev = listLast;
            }
            second.Parent = first;
            second.Mark = false;
            first.Rank++;
            return first;
        }
        private void Consolidate()
        {
            int maxRank = (int)Math.Log(TotalNum, 2) + 1;
            var arrayRanks = new FibonacciNode[maxRank + 100];
            var currRoot = RootlistFirst;
            var last = RootlistFirst.Prev;
            bool lastDone = false;
            do
            {
                if (currRoot == last) { lastDone = true; }
                var nextCurrent = currRoot.Next;
                var rank = currRoot.Rank;
                while (arrayRanks[rank] != null) //node with degree d exists
                {
                    if (MaxDegree < rank + 1) { MaxDegree = rank + 1; }
                    var existingRoot = arrayRanks[rank];
                    if (currRoot.Key > existingRoot.Key)
                    {
                        var temp = currRoot;
                        currRoot = existingRoot;
                        existingRoot = temp;
                    }
                    currRoot = Link(currRoot, existingRoot);
                    arrayRanks[rank] = null;
                    rank++;
                }
                arrayRanks[currRoot.Rank] = currRoot;
                currRoot = nextCurrent;
            } while (!lastDone);
            //find min
            Min = null;
            foreach (var currentRoot in arrayRanks.Where(currentRoot => currentRoot !=
null))
            {
                if (Min == null)
                {
                    RootlistFirst = null;
                    InsertNodeToRootlist(currentRoot);
```

```
            Min = currentRoot;
        }
        else
        {
            InsertNodeToRootlist(currentRoot);
            CheckMin(currentRoot);
        }
    }
    arrayRanks = null;
}
#endregion
public void DecreaseKey(FibonacciNode nodeToDecrease, int decreasedVal)
{
    nodeToDecrease.Key -= decreasedVal;
    var parent = nodeToDecrease.Parent;
    if (parent != null && parent.Key > nodeToDecrease.Key)
    {
        Cut(nodeToDecrease);
        CascadingCut(parent);
    }
    if (nodeToDecrease.Key < Min.Key)
    {
        Min = nodeToDecrease;
    }
}
#region Private Methods for DecreaseKey
private void Cut(FibonacciNode child)
{
    var parent = child.Parent;
    if (parent == null) return;

    //remove child from parent and decrement parent's degree
    DeleteRootFromList(child);
    parent.Rank--;
    //add child to the root list
    InsertNodeToRootlist(child);
}
private void CascadingCut(FibonacciNode node)
{
    var parent = node.Parent;
    if (parent == null) return;

    if (node.Mark == false) node.Mark = true;
    else
    {
        Cut(node);
        CascadingCut(parent);
    }
}
#endregion
public FibonacciNode Delete(FibonacciNode node)
{
    return null;
}
#endregion

#region Test methods
public string GetRootlistKeys()
```

```
    {
        var sb = new StringBuilder();
        var sbRanks = new StringBuilder();
        sb.AppendFormat("Root list: ");
        sbRanks.AppendFormat("Ranks: ");
        if (RootlistFirst == null)
        {
            sb.Append("Empty heap");
        }
        else
        {
            sb.AppendFormat("{0}", RootlistFirst.Key);
            sbRanks.AppendFormat("{0}", RootlistFirst.Rank);
            var current = RootlistFirst.Next;
            while (current != RootlistFirst)
            {
                sb.AppendFormat("-{0}", current.Key);
                sbRanks.AppendFormat("-{0}", current.Rank);
                current = current.Next;
            }
        }
        return sb + ";" + sbRanks;
    }
    public string GetTreeLeaves(FibonacciNode node)
    {
        if (node == null) return "nill";

        var sb = new StringBuilder();
        sb.AppendFormat("Rank {0}: ", node.Rank);
        if (node.Rank == 0)
        {
            var mark = (node.Mark) ? "m" : string.Empty;
            sb.AppendFormat("Leaf {0}{1}", node.Key, mark);
        }
        else if (node.Rank == 1)
        {
            var mark = (node.Mark) ? "m" : string.Empty;
            sb.AppendFormat("{0}{1} - Child: ", node.Key, mark);
            sb.Append(GetTreeLeaves(node.Child));
        }
        else
        {
            sb.AppendFormat("{0} - Children: ", node.Key);
            var firstChild = node.Child;
            if (firstChild != null)
            {
                var current = firstChild;
                do
                {
                    sb.Append(GetTreeLeaves(current));
                    current = current.Next;
                } while (current != firstChild);
            }
        }
        return sb.ToString();
    }
    #endregion
}
```

```
/// <summary>
/// 4 node links props: parent, child, prev, next
/// 1 pointer to arrayObject
/// 1 int props: rank
/// </summary>
public class FibonacciNode
{
    #region Properties
    public FibonacciNode Parent { get; set; }
    public FibonacciNode Child { get; set; }
    public FibonacciNode Prev { get; set; }
    public FibonacciNode Next { get; set; }

    public bool Mark { get; set; }
    public int Rank { get; set; }
    public int Key { get; set; }
    public FibonacciElement Element { get; set; }
    #endregion
}

public class FibonacciElement
{
    public int Index { get; set; }
    public FibonacciNode Node { get; set; }
}
```